What Keeps a Generative Recommender Relevant? An Empirical Study of Reinforcement Fine-Tuning Against a Learned Reward Model

Daniil Antonov

ITMO University, Saint Petersburg, Russia

Draft, version 1 12 October 2026

Abstract

Generative recommenders that produce a list item by item can be fine-tuned with reinforcement learning (RL) towards objectives that next-item training does not see. Published successes keep the reward tied to the log: relevance to the true continuation, or logged feedback on logged recommendations. We study the least protected setting, in which a learned model of user feedback is the only source of reward. A GPT-2 recommender is fine-tuned with REINFORCE and PPO to raise the consumed share of an item predicted by a second GPT-2, on a music log (Yambda-50M) and a video log (KION). A reward model fitted on disjoint users, relevance measured separately on items the user later finished and abandoned, and measures of list concentration separate real gains from exploitation. Without a constraint the policy always collapses to nearly one list for all users. With a KL anchor to the pretrained recommender the outcome depends on the dataset: on KION the predicted share rises by 23%, confirmed by the held-out model and reproduced with retrained reward models, while relevance on finished items is kept; on Yambda the same anchor weight leaves PPO almost unmoved. The weight at which collapse begins is set by the advantage estimator, not the dataset: at one and the same weight REINFORCE loses about 90% of NDCG@10 and PPO none. A logged-relevance reward keeps hits without any anchor but cuts catalogue coverage fivefold, and an auxiliary choice head meant to teach the reward model relevance fails. Returning within a week is predictable yet barely depends on the item shown, so rewarding it buys little. Re-ranking the pretrained recommender's candidates with the same reward model matches RL up to a reward gain of about one third. All results are offline and single-seed.

Keywords: generative recommendation, sequential recommendation, reinforcement learning, reward model, reward hacking, user retention, offline evaluation.

1 Introduction

Sequential recommenders built as autoregressive transformers over item or semantic identifiers [1], [2], [3], [4], [5] are trained to continue logged sequences. This objective imitates the log, and the log records what users did rather than what served them: in the two logs studied here, 36% of music listens stop before half of the track and 52% of video titles are abandoned before half of their length. A recommender that generates a list item by item is also a policy that takes a sequence of decisions, so reinforcement learning (RL) is the natural tool for moving it from "what comes next" towards "what will be received well" [6], [7], [8], [9].

Where the reward comes from decides how safe that step is. Two recent studies that report gains without losing relevance both keep the reward tied to the log. Petrov and Macdonald [8] fine-tune GPTRec with PPO on its own generated lists, with a reward that contains NDCG against the user's true next item next to a diversity or popularity term. Matveev et al. [9] train a retriever with off-policy REINFORCE on logged sessions, with logged feedback as the reward, and use a learned user feedback model only to evaluate. A third option is to let a learned model of user feedback *be* the reward. It is attractive because such a model scores any item for any user: the policy can be trained on its own generations and for feedback signals that the log holds only for the items users chose. It is also the option with the fewest protections. Model-based offline RL [10], [11], [12] and reward-model fine-tuning of language models [13], [14], [15] both document that a policy optimised against a learned model drifts to where the model is wrong, and in recommendation the drift is known to take the form of popularity and filter-bubble effects [16], [17], [18].

This paper studies the third option in its barest form. A GPT-2 [19] next-item recommender is fine-tuned with policy gradients to raise the share of an item that a second GPT-2, the *reward model*, predicts the user will consume. Nothing in this reward asks for relevance; a KL penalty towards the pretrained recommender, which we call the *anchor*, is the only term that does. We ask five questions.

RQ1 Does fine-tuning raise the reward, and how does the behaviour of the recommender change?
RQ2 Is relevance preserved, do users still receive different lists, and which mechanism preserves either?
RQ3 Can longer-term signals, namely continuing the session and returning, serve as rewards?
RQ4 What happens along a list that is generated several steps ahead?
RQ5 What is the trade-off between reward and relevance, and does reaching it take RL?

The study covers two logs that differ in what makes the task hard (Yambda-50M: music, long histories with repeats; KION: video, short histories, one row per user and title), two estimators of the policy gradient (REINFORCE with a leave-one-out baseline, and PPO with a learned value model), and five completed runs of the full pipeline. Because every reward gain is reported by a model, the pipeline carries three instruments that do not depend on the model being optimised: a second reward model fitted on a disjoint half of the users that never influences training; relevance measured separately against future items the user consumed and abandoned; and measures of how concentrated the lists are across users. Our findings are the following.

- **Unconstrained optimisation always collapses.** On both datasets and with both estimators, a policy without an anchor ends up serving nearly the same list to everyone. With a weak anchor on Yambda it moves to a handful of obscure tracks whose few listeners finished them: the optimised reward model reports +40%, the held-out one +16%, and NDCG@10 falls by 89%. The reward model was accurate and calibrated on held-out events (AUC 0.87); that did not protect it (Section 5.2).
- **An anchor that holds gives a dataset-specific result.** On KION the predicted share rises by 23%, the held-out model confirms it, relevance on items the user went on to finish is unchanged (+1%) and recall of abandoned items falls by 20%; the result is reproduced with retrained reward models. On Yambda the same weight under PPO changes almost nothing (Section 5.2, Section 5.3).
- **The anchor weight is not a transferable quantity.** On the same data and at the same weight, REINFORCE loses about 90% of NDCG@10 and PPO loses nothing. We attribute this, as a hypothesis, to advantage normalisation combined with different baselines (Section 5.3).
- **Each place for the relevance requirement protects something different.** The anchor preserves the distribution of the pretrained recommender. A logged-relevance reward [8] preserves hits even with no anchor, and together with the predicted share it beats the anchor at equal reward, but catalogue coverage falls from 0.50 to 0.10. Learning only from logged actions [9] preserves both and gains less. A fourth mechanism, a choice head inside the reward model, fails: among the recommender's candidates it knows far less about relevance than the recommender itself (Section 5.5, Section 5.8).
- **Retention signals are predictable but offer little leverage.** Returning within a week is predicted with AUC 0.70–0.72, yet the signal varies four times less across one user's candidates than across users. Rewarding it raises the predicted return probability by 1.9 points for 7% of NDCG@10; rewarding session continuation promotes titles that users abandon (Section 5.6).
- **The multi-step predicted reward measures the simulation as much as the recommendations.** The reward predicted for the tenth generated item moves by 25 to 35 points with the assumption about the feedback on the nine before it (Section 5.7).
- **A moderate gain does not need RL.** Re-ranking the pretrained recommender's 100 best candidates with the same reward model is at least as relevant as RL up to a reward gain of about one third; supervised fine-tuning on liked items and off-policy REINFORCE land on the main RL run (Section 5.8).

All results are offline, and every configuration was trained once. Section 7 lists what this leaves open.

2 Related Work

Generative and sequential recommenders. The recurrent GRU4Rec [20] and the self-attentive SASRec [1] and BERT4Rec [21] made next-item prediction the default formulation of sequential recommendation. Generative recommenders use the same models autoregressively: GPTRec [2] generates a list item by item (Next-K) instead of scoring the catalogue once (Top-K), TIGER [3] generates semantic identifiers, and industrial systems scale the idea to whole retrieval and ranking stacks [4], [5], [22]. We keep the simplest

member of the family, a GPT-2 decoder over item-identifier tokens, so that the effects of fine-tuning are not confounded with tokenisation.

RL for recommendation. Policy-gradient recommenders trained from logs correct for the logging policy with importance weights [6]; value-based methods decompose the value of a slate [7] or act in a learned slate space [23]; SQN and SA2C attach RL heads to a supervised sequential model [24], [25]. The two studies we build on sit at opposite ends: on-policy PPO over the recommender's own generations with a reward read from the true next item [8], and off-policy REINFORCE with a multi-step correction over logged sessions [9]. Learned models enter rewards when the wanted signal is sparse, as with imputed satisfaction responses [26], and they replace the user altogether in simulators [27], [28], [29]. Our setting is the second case with a simulator learned from the same log the recommender was trained on.

Learned rewards and their exploitation. Model-based offline RL treats the problem as one of trusting a model only where data supports it: MOPO penalises the reward by model uncertainty [10], MOReL builds a pessimistic MDP [11], and MBPO keeps model rollouts short [12]. For language models fine-tuned against learned reward models, Gao et al. [13] show that a "gold" reward first rises and then falls while the proxy keeps rising as the policy moves away from its initialisation, Coste et al. [14] mitigate this with ensembles, and Skalse et al. [15] formalise reward hacking. Our held-out reward model plays the part of the gold model at no extra scale: same architecture, other users. In recommendation, conservative offline RL has been shown to amplify the Matthew effect [16], filter bubbles have been studied as a long-term effect of interactive recommendation [17], and feedback loops have been shown to degenerate [18].

Staying close to a reference policy. TRPO and PPO bound each update relative to the previous iterate [30], [31]. Penalising divergence from a *pretrained* model is the older idea of KL-control [32], [33], used in fine-tuning language models from human preferences [34], [35] and, as behaviour regularisation, in offline RL [36]. Our anchor belongs to this line. What we add is evidence that its weight has no meaning across advantage estimators.

Long-term signals and consumption-time rewards. RL has been applied to long-term engagement [37] and directly to retention [38]; delayed outcomes are commonly approached through proxies [39], and measuring the long-term value of exploration takes dedicated experiment designs [40]. Consumption time is confounded by item duration [41], [42]. We observe the same confounding from the policy's side: the policy moves along the duration axis in whichever direction the normalisation of the reward favours.

Multi-step generation. Feeding a model its own outputs takes it away from the inputs it was trained on [43], [44]; Petrov and Macdonald report that Next-K generation is worse than Top-K ranking after supervised training [8]. Spending inference compute on several candidates that a learned model judges is the recommender form of test-time scaling [9], [45].

Evaluation. Next-item protocols hide deficiencies of RL recommenders [46], and off-policy estimators such as the doubly robust one [47] need logged propensities, which neither of our logs provides. We therefore rely on a held-out model and on model-free checks, and repeat the caveat of [9] that results selected and judged by the same feedback model are not independent evidence. The baselines are trained under one protocol, following the concerns of [48].

3 Method

Figure 1 shows the pipeline: a recommender is pretrained on next-item prediction, two reward models are fitted on disjoint halves of the users, and the recommender is fine-tuned on the reward that one of them predicts for its own generated lists.

3.1 Setting and notation

A log is a set of events, each with a user, an item i_t, a time and the *consumed share* $\rho_t \in [0, 1]$: the share of the track that was played or of the title that was watched. A recommender is a distribution $\pi_\theta(j \mid h)$ over the next item given the history h; π_0 is the recommender before fine-tuning. A recommendation is a list $a_{1:K}$ with $K = 10$. For evaluation, F is the set of the next $M = 20$ items the user really consumed after h, split into *liked* items F^+ with $\rho \geq 0.5$ and *skipped* items F^-.

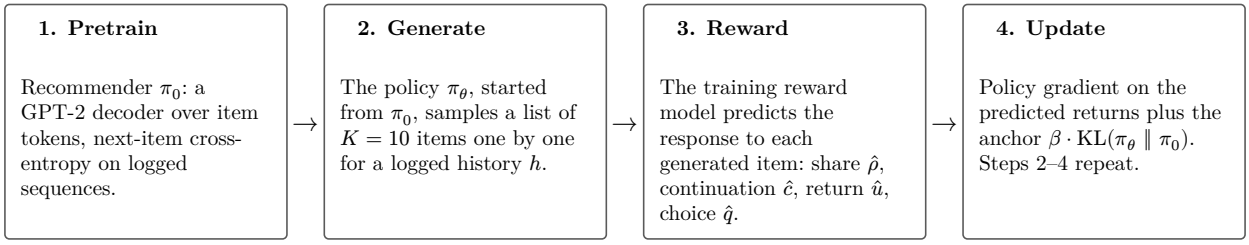


Figure 1: The pipeline. The reward model in step 3 is the only source of reward during fine-tuning; the instruments in the dashed box never influence the policy.

3.2 Base recommender

The recommender is a GPT-2 decoder [19] over item tokens with tied input and output embeddings. Its state s_t at position t summarises $i_{1:t}$, and $\pi_\theta(j \mid i_{1:t}) = \mathrm{softmax}_j(Es_t)$ with the embedding matrix E. It is trained with the full-softmax cross-entropy

$$\mathcal{L}_{\text{next}}(\theta) = -\sum_t \log \pi_\theta(i_{t+1} \mid i_{1:t}). \tag{1}$$

A list is obtained in one of two ways [2]. *Ranking* (Top-K) scores the catalogue once and takes the K best items. *Generation* (Next-K) appends each chosen item to the history as if the user had consumed it, $a_k = \arg\max_j \pi_\theta(j \mid h, a_{1:k-1})$, and never repeats an item within the list. Fine-tuning changes the second mode, in which the recommender is a policy over a sequence of decisions.

3.3 Reward model

The reward model is a second GPT-2 decoder of the same size. The token at position t carries the item i_t together with the consumed share of the *previous* item, ρ_{t-1}, in ten buckets. The state at position t has therefore seen the candidate item and the whole history with its responses, but not the response it has to predict. One forward pass scores every event of a real history, and a generated item can be scored before any outcome for it exists. Four heads read this state.

- $\hat{\rho}$, the consumed share, trained with binary cross-entropy against ρ as a soft label.
- $\hat{c}$, the probability that the session continues: the user's next event follows within a gap g.
- $\hat{u}$, the probability that the user returns: a new session starts within a window w.
- $\hat{q}$, the probability that the user would choose the item at all.

The first two heads are present in all runs; the last two were added for the reward study (run E, Section 4).

Retention labels. For an event at time τ and later events of the same user at times τ',

$$c = \mathbb{1}\Big[\min_{\tau'}(\tau' - \tau) \le g\Big], \quad u = \mathbb{1}[\exists \tau' : g < \tau' - \tau \le w]. \tag{2}$$

When nothing follows an event, either the user did not come back or the log ended before the window closed. We treat an outcome as known only if the whole waiting time lies inside the observed data, $\tau + w \le T$ for a log that ends at T; unknown outcomes take part in neither training nor evaluation.

Choice head. The log holds only items that users picked, so $\hat{\rho}$ estimates the share *given that the user is consuming the item* and says nothing about an item this user would never pick. To give the reward model a notion of choice, one item per training window is replaced by the item of another event in the batch, and $\hat{q}$ is trained to tell logged items from replacements:

$$\mathcal{L}_q = -\frac{1}{2}\mathbb{E}[\log \hat{q}(i_t \mid h_t)] - \frac{1}{2}\mathbb{E}_{j\sim p}[\log(1 - \hat{q}(j \mid h_t))]. \tag{3}$$

Replacements follow the popularity distribution p, so popularity alone cannot separate them. As in noise-contrastive estimation [49], the optimum is $\hat{q}^*(j \mid h) = P(j \mid h)/(P(j \mid h) + p(j))$: above one half when the item is more likely for this user than on average.

Two copies. Two reward models are trained on disjoint halves of the users. The *training* reward model produces the rewards during fine-tuning. The *held-out* reward model never influences the policy and only scores its lists. A gain that the held-out model does not confirm is exploitation of the errors of the training model.

3.4 Reinforcement fine-tuning

One episode is the generation of one list. The state is $s_k = (h, a_{1:k-1})$, the action is the next item a_k, the reward r_k is predicted by the training reward model (by default $r_k = \hat{\rho}_k$), and the return is $G_k = \sum_{l \geq k} \gamma^{l-k} r_l$. No real user is in the loop: the reward model is the environment. To score the k-th item it needs the responses to the items generated before it, which nobody observed; by default it is fed its own expectation $\hat{\rho}$ for them. Section 5.7 examines this assumption.

Histories are sampled from the training period as a user and a cut-off point. The policy gradient is estimated as $\mathbb{E}\left[\sum_k A_k \nabla_\theta \log \pi_\theta(a_k \mid s_k)\right]$ with one of two advantage estimators.

REINFORCE with a leave-one-out baseline. Several lists are sampled for each history, and the baseline of a list is the mean return of the other lists sampled for the same history [50], [51], which leaves the gradient unbiased and removes the part of the return that depends on the user alone.

PPO with a learned value model. A value model V_φ, a copy of the recommender's transformer with a scalar head, predicts the return from every position. Advantages are generalised advantage estimates [52], $A_k = \sum_{l \geq 0} (\gamma\lambda)^l \delta_{k+l}$ with $\delta_k = r_k + \gamma V_\varphi(s_{k+1}) - V_\varphi(s_k)$, and each batch of lists is used for several steps on the clipped surrogate [31]

$$\mathcal{L}_{\mathrm{PPO}}(\theta) = -\mathbb{E}[\min(w_k A_k, \mathrm{clip}(w_k, 1-\varepsilon, 1+\varepsilon) A_k)], \quad w_k = \frac{\pi_\theta(a_k \mid s_k)}{\pi_{\theta_{\mathrm{old}}}(a_k \mid s_k)}. \tag{4}$$

In both cases advantages are scaled to zero mean and unit variance within the batch, and dropout is switched off during fine-tuning so that recorded and recomputed log-probabilities agree.

Anchor. The full loss adds a penalty on the divergence from the frozen pretrained recommender, computed exactly over the catalogue at every position:

$$\mathcal{L}(\theta) = \mathcal{L}_{\mathrm{PG}}(\theta) + \beta \cdot \mathbb{E}_k[\mathrm{KL}(\pi_\theta(\cdot \mid s_k) \parallel \pi_0(\cdot \mid s_k))], \tag{5}$$

where $\mathcal{L}_{\mathrm{PG}}$ is the REINFORCE or the PPO term. Since the default reward has no relevance term, the weight β is the only thing that asks the policy to stay relevant.

3.5 Reward family

All rewards studied here are instances of

$$r_k = \hat{q}_k^b \left(w_\rho \hat{\rho}_k + w_c \hat{c}_k + w_u \hat{u}_k\right) + w_F \frac{\mathbb{1}[a_k \in F]}{\log_2(k+1)}, \quad b \in \{0, 1\}. \tag{6}$$

The default is $w_\rho = 1$ with everything else at zero. The *time* variant multiplies $\hat{\rho}_k$ by the duration of the item relative to the median duration in the catalogue. The last term is the *logged-relevance* reward of Petrov and Macdonald [8]: for a history sampled from the training period the user's real continuation F is known, and the sum of this term over the list is DCG@K. With $b = 1$ the model-predicted part is weighted by the choice probability, so that an item the user would not pick earns little however high its average consumed share. The two mechanisms put the relevance requirement *inside* the reward, where the anchor puts it outside.

3.6 Alternatives to on-policy RL

Four methods raise the same reward without generating lists in a training loop.

- **Liked-only fine-tuning** continues supervised next-item training for one epoch, on targets with $\rho \geq 0.5$ only. It uses logged feedback and no reward model.
- **Off-policy REINFORCE** on logged sessions, after [9]. Nothing is generated and no reward model is asked: the log-probability of every logged item is raised in proportion to the logged reward that followed it,

$$\mathcal{L}(\theta) = -\frac{\sum_t \gamma^{n_t} \bar{\omega}_t G_t \log \pi_\theta(i_t \mid i_{1:t-1})}{\sum_t \gamma^{n_t} \bar{\omega}_t}, \quad \bar{\omega}_t = \min\left(\kappa, \prod_{l=t}^{t+L-1} \frac{\pi_\theta(i_l \mid i_{1:l-1})}{\pi_0(i_l \mid i_{1:l-1})}\right), \tag{7}$$

where G_t is the discounted sum of logged rewards to the end of the session, n_t the position of the action in its session, and the frozen pretrained recommender stands for the behaviour policy. The cap κ on the importance weights limits how far the policy may move. We use $\gamma = 0.9$ and $L = 10$. Unlike the original, every logged event counts as an action, because KION does not mark which events a recommender served.

- **Re-ranking** takes the 100 best candidates of the pretrained recommender and keeps the K with the highest $r(j \mid h) + w \log \pi_0(j \mid h)$, where r is the reward predicted for j as the next item after the real history. Nothing is trained; the weight w plays the part of the anchor.
- **Best of n sampled lists** samples n lists from the pretrained recommender and serves the one with the highest predicted return, the test-time scaling of [9] applied to whole lists.

4 Experimental Setup

4.1 Data

Yambda-50M [53] is a listening log of Yandex Music: 46.5 million listens of 10,000 users over about 300 days. Each event has the track, a timestamp, the played share of the track, the track length and a flag telling whether the user chose the track or a recommender served it. **KION** [54] is a log of the MTS online cinema: 5.5 million rows of 962 thousand users over about five months. It has one row per user and title, with the date of the last viewing, the total watch time and the watched share of the title. Time is given as dates, and nothing says whether a title was recommended.

Both logs are split along one time axis: the last 7 days are the test period, the 7 days before them the validation period, the rest the training period. The catalogue, popularity and all item statistics come from the training period only. On Yambda the 50,000 most popular training-period tracks become tokens, numbered by popularity, events on other tracks are dropped, and a user needs at least 50 training events. On KION a title needs 10 training events and a user 5. Table 1 compares the processed logs.

The consumed share is bimodal in both logs (Figure 9 in the appendix): an item is either dropped early or finished, so the reward model is in effect a skip classifier. The two logs differ in how the share depends on the length of the item. On Yambda short tracks are finished somewhat more often than long ones, 0.70 against 0.60 between the extreme duration deciles. On KION titles of film length, roughly 80 to 140 minutes, are watched to 0.5 on average, while the longest tenth of the catalogue, more than 8.5 hours of content and hence series, is watched to 0.22, because the share is taken of the whole series. A reward built on this share prefers films before any model is involved.

Retention signals. In the first four runs there is one signal: the event "continues the session" if the same user has another event within 30 minutes (Yambda; true for 95% of events) or watches another title within 7 days (KION; 80%). For the reward study on KION (run E) it is split into two. A session is one calendar day; *session continuation* holds for 42% of events and *return*, a viewing on another day within a week, for 71%. The outcome of return is known for 89% of events. The order of a KION user's titles within a day is the order of rows in the file, so on KION session continuation says how many titles the day had more than which title kept the viewer.

Table 1: The two processed logs.

	Yambda-50M	**KION**
Events kept	37.2 M (80.1%)	3.88 M (70.9%)
Users	8,711	256,407
Items in the catalogue	50,000	7,992
Events per user, mean	4,270	15
Training / validation / test events	35.0 M / 1.13 M / 1.08 M	3.51 M / 0.19 M / 0.18 M
Mean consumed share	0.65	0.50
Events consumed at least half	64%	48%
Test users with a request	7,593	59,997

4.2 Models and training

The recommender has 4 layers, width 256 and 4 attention heads. On Yambda it is trained on windows of 128 consecutive events with dropout 0.1; on KION on windows of 48 events with dropout 0.2. Optimisation uses AdamW [55] with learning rate 10^{-3}, cosine decay and early stopping on the validation loss. Re-recommending items from the history is allowed on Yambda, where repeated listening is normal, and blocked on KION, where every user–title pair occurs once; baselines and re-ranking obey the same constraints. The reward models have the same size.

REINFORCE (runs A and B) samples 4 lists per history and uses a learning rate of $3 \cdot 10^{-5}$. PPO (runs C to E) samples one list for each of 512 histories per iteration and takes four gradient steps on them with clipping threshold $\varepsilon = 0.2$, discount $\gamma = 0.99$ and $\lambda = 0.9$. The learning rates are 10^{-5} for the policy and 10^{-4} for the value model, which is trained alone for the first 50 iterations.

4.3 Evaluation

Each test user with a history makes one request at the start of the test week. The last 118 events (38 on KION) are the history and the next at most 20 events are the ground truth. This gives 7,593 requests on Yambda and 59,997 on KION. Three groups of quantities are measured on lists of ten.

- **Relevance.** Recall, NDCG, MAP and HitRate at 1, 5 and 10 against all future items, against F^+ (*liked*) and against F^- (*skipped*). The split needs no reward model: a policy that has really learned to avoid what users abandon should lose relevance on skipped items and keep it on liked ones.
- **Predicted reward.** Both reward models score the generated lists.
- **Concentration.** Catalogue coverage (the share of items that appear in at least one list), the share of users who receive the most frequent item (*top-item share*), the expected overlap between the lists of two users (*pair overlap*), the share of recommendations already in the history, and the popularity and duration of recommended items.

In the PPO runs the change from the pretrained to the fine-tuned recommender is reported with a 95% interval from a paired bootstrap over users. It measures the noise of the user sample, not the noise between training runs.

4.4 Baselines

The pretrained recommender is compared with the most popular items, a first-order Markov chain, GRU4Rec with the full softmax [20], SASRec with its original loss (binary cross-entropy against one sampled negative per position) [1] and BERT4Rec trained on the cloze task [21]. The neural baselines take the width, depth, dropout and window length of the recommender, are trained on the same windows and are stopped by their own validation loss. None of them was tuned.

4.5 Runs

The pipeline was completed five times (Table 11 in the appendix). Runs A and B use REINFORCE on Yambda; B is a reduced-budget version on a much weaker GPU with a 30,000-track catalogue. Run C uses PPO on KION and is the first with baselines, alternatives to RL and intervals. Run D repeats the pipeline of run C on the data of run A and so separates the dataset from the estimator. Run E repeats run C with newly trained reward models and adds the reward study of Section 5.5 and Section 5.6. The anchor weight β is swept in every run, and the *main* RL run of each uses a fixed weight: 0.1 in runs A and B, 0.3 in runs C to E.

5 Results

5.1 The pretrained models are sound

Recommender. Table 2 places the pretrained recommender among the baselines. On Yambda it is the best of those compared: 4% above GRU4Rec, 8% above BERT4Rec and 48% above SASRec in NDCG@10 for ranked lists. Two caveats apply: GRU4Rec and BERT4Rec reached the limit of 10 epochs while their validation loss was still falling, and no intervals were computed between baselines. The NDCG@10 of SASRec, 0.077, is close to the 0.075 published for Yambda-50M [53], although the protocols differ.

Table 2: Baselines and the GPT recommender on lists of ten. Every model is trained once and none is tuned.

	Popular	**Markov**	**GRU4Rec**	**SASRec**	**BERT4Rec**	**GPT, ranked**	**GPT, generated**
Yambda-50M, 7,593 test users (run D)							
NDCG@1	0.0246	0.0979	0.1488	0.1030	0.1534	0.1653	0.1653
NDCG@10	0.0186	0.0550	0.1097	0.0767	0.1054	0.1136	0.1165
Recall@10	0.0114	0.0335	0.0718	0.0506	0.0673	0.0737	0.0742
MAP@10	0.0079	0.0272	0.0591	0.0366	0.0585	0.0621	0.0747
HitRate@10	0.1118	0.2612	0.4417	0.3806	0.4163	0.4454	0.3656
Catalogue coverage	0.0002	0.457	0.448	0.397	0.459	0.464	0.519
KION, 59,997 test users (run C)							
NDCG@1	0.0597	0.0669	0.0877	0.0726	0.0945	0.0885	0.0885
NDCG@10	0.0903	0.0959	0.1215	0.1082	0.1290	0.1220	0.1173
Recall@10	0.1303	0.1394	0.1721	0.1580	0.1811	0.1726	0.1620
MAP@10	0.0607	0.0637	0.0824	0.0720	0.0885	0.0831	0.0819
HitRate@10	0.2358	0.2571	0.3154	0.2903	0.3281	0.3144	0.2871
Catalogue coverage	0.004	0.466	0.544	0.335	0.427	0.574	0.495
Top-item share	0.918	0.530	0.502	0.504	0.496	0.484	0.566
Pair overlap	0.714	0.220	0.149	0.161	0.161	0.140	0.209

On KION popularity is a strong floor: the ten most popular unseen titles reach NDCG@10 0.090, and the recommender is 35% above it, against a factor of six on Yambda. The competition for which the dataset was released shows the same picture: its five best solutions scored MAP@10 between 0.114 and 0.122 against 0.091 for popularity [54][1], a ratio of 1.25 to 1.34; ours is 1.37 for the GPT recommender and 1.46 for BERT4Rec. Protocols differ, so only the ratio is comparable. The recommender is mid-table on KION: level with GRU4Rec, 13% above SASRec and 5% below BERT4Rec. Lists are concentrated on KION before any RL: every neural model gives its most frequent title to about half of the users, and two users share 14% to 21% of their lists, against 2% and 0.1% on Yambda. Concentration measures therefore start high on KION.

Ranking against generation. Generating the list item by item does not hurt on Yambda (NDCG@10 0.117 against 0.114 for ranking) and hurts slightly on KION (0.117 against 0.122, with a smaller coverage). The KION result is the one Petrov and Macdonald report [8]. A likely reason for the difference is that Yambda users replay tracks, so a generated track is a plausible part of the history it joins, while a KION title is watched once; in addition our ground truth is the next 20 events and not the single next item.

Reward models. Both reward models predict held-out feedback much better than simple predictors (Table 3). On Yambda they are almost perfectly calibrated. On KION calibration is good in the middle of the range and compressed at the ends: events predicted at about 0.06 are watched to 0.12 and those predicted at about 0.93 to 0.87. The two KION copies agree, with a correlation of 0.94 between their predictions on test events. Three properties matter for what follows.

- *Where the skill comes from.* On Yambda the recent behaviour of the user alone reaches AUC 0.80 and the mean share of the track only 0.61: most of the model's skill is knowing whether this user is skipping right now, and knowledge about individual tracks, the part RL can act on, is the smaller part. On KION the title alone predicts almost as well as the user (0.66 against 0.67): there the reward is to a large extent a property of the title, and a policy can raise it by choosing titles.
- *Where it errs.* The KION models overestimate the watched share of the least popular fifth of the catalogue by 0.13–0.14, against 0.02 for popular titles.
- *Recommended against organic events.* On Yambda the model is more accurate on events served by a recommender (AUC 0.895, MAE 0.198) than on events the user chose (AUC 0.834, MAE 0.273). RL asks the model about recommendations.

5.2 Fine-tuning at three operating points

Table 4 shows the main RL run of three settings. They end in three different places: a collapse, the intended result, and almost no change.

[1]Competition results are listed in the README of `github.com/irsafilo/KION_DATASET`.

Table 3: Prediction of the consumed share on the events of the test period, each from the events before it. AUC is for the event "consumed at least half".

Predictor of the consumed share	**MAE**	**RMSE**	**Cross-entropy**	**AUC**
Yambda-50M (run A)				
Training reward model	0.236	0.327	0.433	0.867
Held-out reward model	0.243	0.328	0.434	0.865
Mean over the user's previous events	0.289	0.370	0.510	0.798
Mean share of the item	0.393	0.429	0.623	0.607
Global mean	0.409	0.434	0.635	0.500
KION, 184,209 test events (run C)				
Training reward model	0.290	0.347	0.608	0.756
Held-out reward model	0.291	0.349	0.610	0.753
Mean over the user's previous events	0.331	0.384	0.670	0.673
Mean share of the item	0.334	0.379	0.657	0.656
Global mean	0.370	0.399	0.689	0.500

Table 4: The main RL run of three settings against the pretrained recommender, on generated lists for test users: run A (4,000 steps), run D and run C (2,000 iterations each). *Share* and *time* are the consumed share and the consumption time that a reward model (RM) predicts for the lists; the KL divergence is per list position.

	Yambda, REINFORCE $\beta = 0.1$			**Yambda, PPO $\beta = 0.3$**			**KION, PPO $\beta = 0.3$**		
	Base	**RL**	**Change**	**Base**	**RL**	**Change**	**Base**	**RL**	**Change**
NDCG@10, all	0.1175	0.0125	−89%	0.1165	0.1190	+2.2%	0.1173	0.1039	−11.4%
NDCG@10, liked	0.1010	0.0123	−88%	0.1012	0.1032	+2.0%	0.1214	0.1228	+1.2%
Recall@10, skipped	0.0737	0.0059	−92%	0.0714	0.0745	+4.4%	0.1382	0.1112	−19.6%
Share, training RM	0.580	0.811	+40%	0.581	0.591	+1.8%	0.433	0.536	+23.9%
Share, held-out RM	0.613	0.713	+16%	0.610	0.617	+1.1%	0.444	0.545	+22.8%
Time, held-out RM (s)	123.1	93.2	−24%	122.3	120.4	−1.6%	6,361	5,632	−11.5%
Catalogue coverage	0.523	0.050		0.519	0.503		0.495	0.434	
Top-item share	0.023	0.821		0.026	0.040		0.566	0.554	
Pair overlap	0.001	0.466		0.0014	0.0022		0.209	0.275	
Mean item duration	203 s	130 s		203 s	197 s		5.7 h	3.2 h	
KL to π_0 (nats)		2.8			0.03			0.10	

A weak anchor: the policy maximises the reward and stops recommending. On Yambda with REINFORCE and $\beta = 0.1$, the predicted share rises and nearly everything else is lost. Four observations explain how.

What it moved to. The fifteen tracks with the most impressions have between 136 and 2,294 plays in the training period (the most popular track has 39,487), last between 90 and 165 seconds, and were finished to 85–97% by their listeners, against 65% overall. The most shown of them is in the lists of 6,235 of 7,593 users. These are niche tracks with a self-selected audience: those who found them like them, and the reward model extends this to everybody. The policy also abandons tracks the user already knows: the share of recommendations from the history falls from 51% to 4%.

Part of the gain is earned on the model's errors. The training reward model credits the new lists with +40%, the held-out one with +16%. The held-out estimate stops rising after about 750 steps while the training estimate keeps rising to the end (Figure 2). Everything gained after that point is seen only by the model being optimised.

The log does not confirm the gain. Relevance falls as much on liked tracks as on skipped ones. The policy has not learned to avoid what users skip; it has left the region the log says anything about.

Held-out accuracy did not protect the reward model. The model is accurate and calibrated on events that users produced. RL asks it about user–track pairs that never occurred, and there its idea of the item on average takes over.

An anchor that holds: the intended result on KION. With PPO and $\beta = 0.3$ the predicted share rises by 23%, and the held-out model reports +22.8% where the training model reports +23.9%; both estimates rise together and flatten after about 500 iterations (Figure 10 in the appendix). The log agrees.

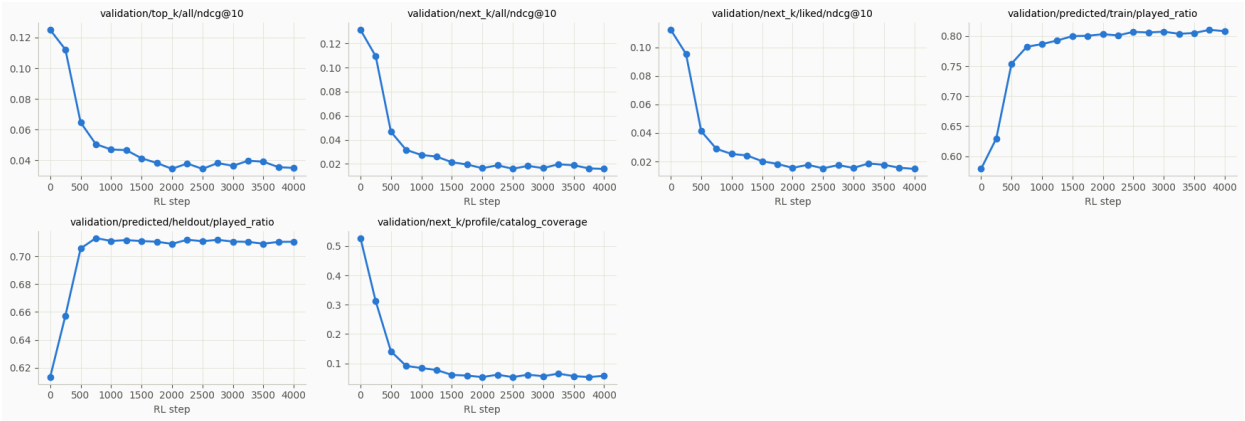


Figure 2: Greedy lists for validation users during the weakly anchored run on Yambda (REINFORCE, $\beta = 0.1$, run A). Top row: NDCG@10 of ranked lists, of generated lists, of generated lists on liked items, and the share predicted by the training reward model. Bottom row: the share predicted by the held-out reward model, and catalogue coverage. Relevance is lost within the first 500 steps; the held-out reward flattens early while the training reward keeps rising.

Relevance on titles the user went on to watch at least half is unchanged, with an interval for the change in NDCG@10 of [0.0000, +0.0028], while recall of abandoned titles falls by a fifth ([−0.0289, −0.0250]) and overall NDCG@10 by 11.4% ([−0.0144, −0.0124]). The policy gives up what users abandon and keeps what they finish. It stays close to the pretrained recommender: the KL divergence settles near 0.10 nats per position, against 2.8 in the collapsed run.

The same weight on Yambda under PPO: almost nothing. The changes lie outside their intervals but are small: +1.1% reward by the held-out model ([+0.0061, +0.0079] in absolute terms) and +2.2% NDCG@10 ([+0.0007, +0.0043]). Relevance rises on liked and on skipped tracks, more on the skipped. The policy has not learned to avoid skips; it has shifted slightly towards popular tracks (mean popularity +12%), which are more often in any continuation. The KL divergence reaches only 0.03 nats per position. The value model explains about 80% of the variance of returns here against about half on KION: on Yambda the reward is mostly determined by the state of the user, not by the track chosen, which leaves the policy little to act on.

How PPO behaved. The share of actions beyond the clipping threshold was zero throughout on KION and 0.01% on Yambda: the small learning rate and the anchor kept every update inside the threshold without clipping. In effect this is an actor–critic method with generalised advantage estimation and four passes over each batch.

5.3 The anchor decides, and its weight does not transfer

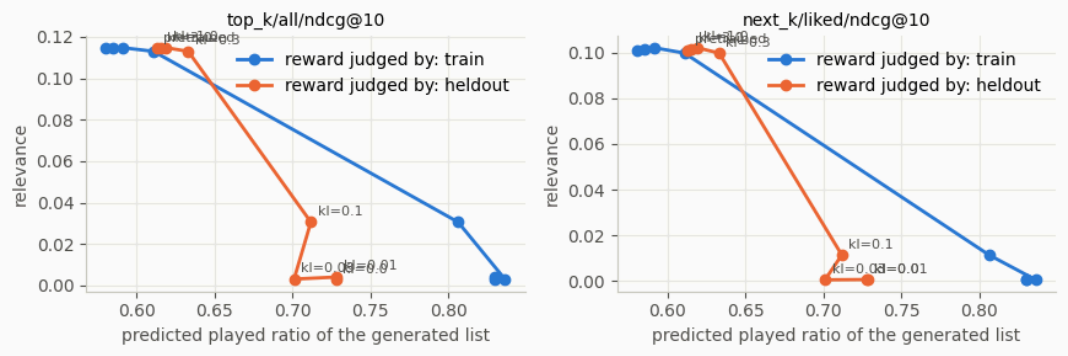
Table 5 compares the three settings at the anchor weights they share, and Figure 3 shows two of the sweeps as trade-off curves; the full sweeps are in Appendix B.

Without an anchor the policy always collapses. On Yambda with REINFORCE all users receive the same ten tracks (coverage 0.0002 of 50,000 tracks, pair overlap 1.0); with PPO, the same twenty or so. On KION only 46 titles are recommended at all, and two users share 99% of their lists. PPO's clipped objective is no protection: without the anchor PPO collapses exactly as REINFORCE does.

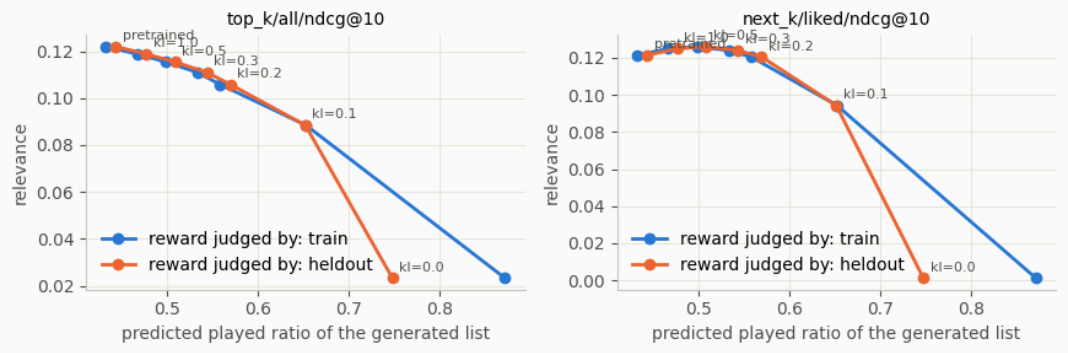
On KION the trade-off is smooth. Every loosening of the anchor buys predicted reward with relevance, and no step is a cliff. From weight 1 down to 0.2 the reward gain grows from 7% to 28% while relevance

Table 5: Change relative to the pretrained recommender in the anchor sweeps: run A (2,000 steps), run D and run C (1,000 iterations each). NDCG@10 on all future items of generated lists; reward is the share predicted by the held-out reward model.

	Yambda, REINFORCE		Yambda, PPO		KION, PPO	
Anchor weight β	**NDCG@10**	**Reward**	**NDCG@10**	**Reward**	**NDCG@10**	**Reward**
0	−99%	+19%	−99%	+18%	−99%	+68%
0.1	−90%	+16%	+0.7%	+2.5%	−37%	+47%
0.3	−1.7%	+3.3%	+0.9%	+1.1%	−11%	+23%
1	+0.7%	+1.0%	+0.6%	+0.4%	−1.7%	+7%



(a) Yambda, REINFORCE (run A)



(b) KION, PPO (run C)

Figure 3: Relevance against predicted reward in two anchor sweeps; each point is one anchor weight. Left panels: NDCG@10 of ranked lists on all future items. Right panels: NDCG@10 of generated lists on liked items. The horizontal axis is the mean predicted share of the generated list according to the training reward model (blue) and the held-out one (orange); the horizontal distance between the curves is reward that only the optimised model reports.

on liked titles stays within 4% of the pretrained recommender, mostly above it; what is lost is recall of abandoned titles, from 2% to 27%. Below 0.2 liked titles go as well: at 0.1 the reward is 47% higher, NDCG@10 on liked titles 22% lower and overall NDCG@10 37% lower. The two reward models agree at every weight except zero, where the training model reports 0.87 and the held-out model 0.75. Exploitation of the reward model appears only at the edge.

On Yambda with REINFORCE the transition is abrupt. Below 0.1 the policy degenerates completely, at 1 and above it barely moves, and the interesting behaviour lies between 0.1 and 0.3, a range the grid stepped over. At 0.3 the result is what RL is meant to deliver, in small: NDCG@10 falls by 1.7%, NDCG@10 on liked tracks by 1.2% and recall of skipped tracks by 6.2%, so the policy gives up more of what is skipped than of what is played, and the predicted share rises by 5.3% according to the training model and 3.3% according to the held-out one.

The collapse threshold depends on the advantage estimator. Run D repeats the sweep on Yambda with PPO. At $\beta = 0.1$, where REINFORCE lost 90% of NDCG@10, PPO loses nothing; at 0.3 PPO moves the reward three times less. The abrupt transition between 0.1 and 0.3 is therefore a property of that setting and not of Yambda, and for PPO on Yambda the whole grid from 0.1 to 1 lies on the safe side: the transition is somewhere between 0 and 0.1.

The most likely explanation is advantage normalisation. In both algorithms advantages are scaled to unit variance, so the size of the policy-gradient term is fixed and the anchor weight is measured against it. With the leave-one-out baseline the contribution of the user is subtracted exactly, and the normalised advantage consists entirely of differences between actions. With PPO the baseline comes from a value model that explains 80% of the variance on Yambda; the remaining 20% is a mixture of differences between actions and errors of the value model. After normalisation the useful signal takes a smaller part of the advantage, and the same weight holds the policy more firmly. This is a hypothesis: the learning rate (three times lower) and the number of lists per step changed together with the estimator. The direct test is PPO with the leave-one-out baseline at the same weights.

The practical consequence is that an anchor weight cannot be carried from one setting to another, and that runs are better placed by the KL divergence they ended at than by the weight that produced it. Nor can the milder outcome on KION at $\beta = 0.1$ (-37% against -90%) be credited to PPO: the dataset, the learning rate and the number of updates changed with it.

The KION result is reproduced with new reward models. Run E retrains both reward models and repeats the main run with the same recommender (Table 16 in the appendix). NDCG@10 is again 0.1039 (-11.4%), NDCG@10 on liked titles $+1.3\%$ with an interval that now excludes zero ($[+0.0003, +0.0030]$), recall of abandoned titles -20.2%, and the predicted share $+24.0\%$ and $+23.1\%$ by the two models.

5.4 Every reward is gamed through the attribute it correlates with

The policy follows whichever property of items the reward correlates with (reward variants are tabulated in Appendix C).

Consumed share. Under a weak anchor on Yambda the policy moves to short obscure tracks (mean duration from 203 to 130 seconds). On KION, with an anchor that holds, it moves from series to films. The titles that gain the most impressions are films of 85 to 130 minutes with a mean watched share of 0.54 to 0.76; the largest gainer goes from the lists of 8,388 users to 26,169 of 59,997. The largest losers are series: one with 17 hours of content and a mean share of 0.08 falls from 33,948 lists to 8,045, another from 16,278 to 558. The mean duration of a recommended title falls from 5.7 hours to 3.2. Two niche films with 953 and 175 viewers in the training period also rise, to 20,403 and 15,901 lists: the Yambda pattern in a small dose.

This qualifies the KION result of Section 5.2. Because the watched share of a series is taken of the whole series, a series is rarely "finished" by the one-half rule and scores low for everybody. Both the reward gain and the divergence between liked and skipped relevance partly restate one fact, that the policy now prefers films, and the predicted watch *time* falls by 11.5%. The reward model is not wrong here; the reward asks for the wrong thing. Whether a finished film is worth more than a started series is a product question that the consumed share answers by construction.

Time. The time variant was meant to remove the preference for short items. It overshoots in the other direction on both datasets. On Yambda at $\beta = 0.3$ the mean duration of recommended tracks rises from 203 to 710 seconds, the predicted listening time triples and NDCG@10 falls by 22%. On KION the mean duration of a recommended title rises from 5.7 hours to 64, the predicted watch time grows sixfold, and the predicted share, the predicted continuation and a quarter of the relevance are lost.

Session continuation. On Yambda a reward for continuing the session moves nothing (predicted continuation 0.9371 against 0.9372): with 95% of events continuing the session, the signal has nowhere to go. The retention signals of KION are examined in Section 5.6.

5.5 Relevance inside the reward: logged relevance and the choice head

If the reward itself asked for relevance, the anchor might not be needed. Run E tests the two mechanisms of Section 3.5 on KION (Table 6, Figure 4).

Table 6: Rewards with their own relevance term on KION (run E), 1,000 PPO iterations each. The rows with the plain share are the anchor sweep of the same run.

Reward	β	**NDCG@10, all**	**NDCG@10, liked**	**Recall@10, skipped**	**Share (held-out)**	**Coverage**	**Pair overlap**
Pretrained recommender		0.1173	0.1214	0.1382	0.459	0.495	0.209
Share $\hat{\rho}$	0.3	0.1048	0.1239	0.1123	0.562	0.434	0.260
	0.1	0.0767	0.0980	0.0686	0.645	0.274	0.284
	0	0.0010	0.0015	0.0005	0.743	0.010	0.987
Choice-weighted share $\hat{q} \cdot \hat{\rho}$	0.3	0.0994	0.1141	0.1105	0.539	0.447	0.156
	0.1	0.0352	0.0465	0.0244	0.672	0.238	0.477
	0	0.0004	0.0006	0.0002	0.687	0.004	0.999
Logged relevance only	0	0.1180	0.1279	0.1417	0.474	0.097	0.519
Share + 10 × logged relevance	0	0.0971	0.1229	0.0843	0.640	0.103	0.577
Share + 3 × logged relevance	0	0.0749	0.0970	0.0622	0.698	0.088	0.498
Share + 1 × logged relevance	0	0.0088	0.0137	0.0035	0.718	0.013	0.964
Share + 0.3 × logged relevance	0	0.0042	0.0066	0.0020	0.714	0.012	0.984

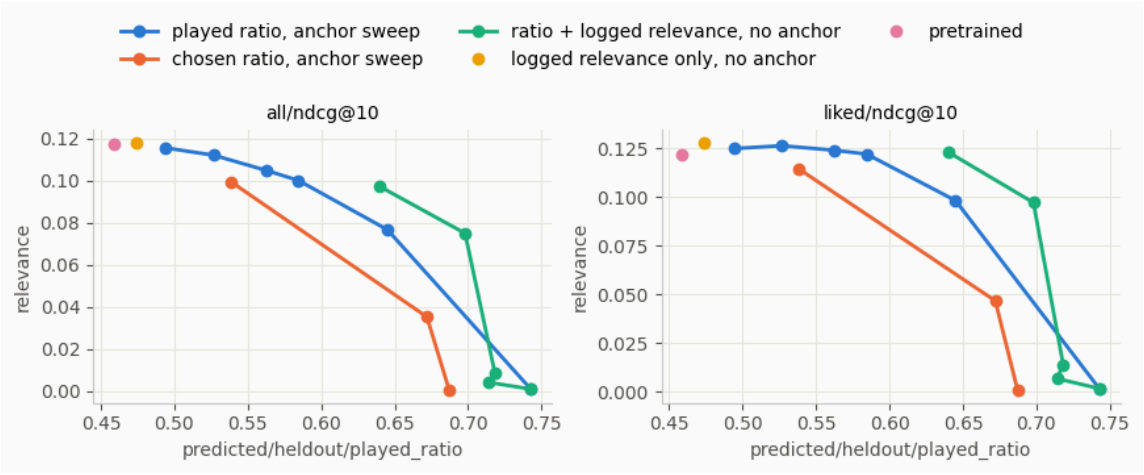


Figure 4: Relevance against the share predicted by the held-out reward model on KION (run E), on all future items (left) and on liked items (right). Blue: the plain share under the anchor sweep. Orange: the choice-weighted share under the anchor sweep. Green: the share plus logged relevance at four weights, with no anchor. Yellow: logged relevance alone, no anchor. Pink: the pretrained recommender.

Logged relevance alone keeps relevance with no anchor at all. NDCG@10 changes by +0.6% and NDCG@10 on liked titles by +5.4%. There is no notable gain in NDCG, as in [8], where such fine-tuning matched the NDCG of supervised training. The lists, however, contract onto popular titles: coverage falls from 0.50 to 0.10, the pair overlap rises from 0.21 to 0.52, and the most popular 1% of titles now receive 91% of recommendations instead of 57%. Relevance metrics do not see this.

Added to the predicted share, it keeps relevance better than the anchor. With weight 10 the predicted share rises by 39% while NDCG@10 on liked titles is unchanged (+1%) and overall NDCG@10 falls by 17%. The anchor at the same reward ($\beta = 0.1$, +41%) loses 19% on liked titles and 35% overall. With weight 3 the reward rises by 52% for a loss of 36% of NDCG@10, a region the anchor does not reach at all. At weight 1 and below the relevance term no longer holds and the policy collapses.

It does not keep diversity. In every run with logged relevance the coverage is about 0.10, against 0.27 for the anchor at the same reward. The anchor preserves the whole distribution of the pretrained recommender, its diversity included; the logged-relevance reward preserves only hits, and the easiest way to hit is to be popular. The two mechanisms are complementary.

The choice head separates swaps but does not know relevance. The head does the task it was trained on: its AUC for a logged item against a replacement is 0.70 for the training reward model and 0.68 for the held-out one, with mean scores of 0.70 and 0.53. As a relevance signal it fails, which shows in four ways.

- *Among the recommender's candidates.* The probability that the candidate the user really consumed next scores higher than the others is 0.56 for $\hat{q}$ and 0.70 for the recommender's own log-probability (Table 7). The recommender knows far more about what suits the user.
- *On collapsed policies.* A policy without an anchor and with NDCG@10 0.001 receives $\hat{q} \cdot \hat{\rho} = 0.53$ from the held-out model against 0.34 for the pretrained recommender; $\hat{q}$ itself falls only from 0.74 to 0.70. The quantity that was meant to fall when relevance is lost rises.
- *On the most relevant policy.* The logged-relevance policies, which keep NDCG@10, receive the lowest $\hat{q}$: between 0.60 and 0.71.
- *Along the list.* $\hat{q}$ rises from 0.73 at the first position to 0.80 at the tenth, while the probability of being in the real continuation falls from 0.088 to 0.016 (Figure 6).

The cause is probably the construction of the negatives. Replacements are drawn by popularity, so $\hat{q}$ measures how much more likely an item is for this user than on average, and a popular title scores about one half whoever watches it. On KION relevance is to a large extent popularity: popular items alone reach 0.090 of the recommender's 0.122. The head penalises exactly what is most often relevant. Its training also had not converged: its validation AUC rose over all eight epochs, from 0.63 to 0.72, while training was stopped on the total loss. Fine-tuning on $\hat{q} \cdot \hat{\rho}$ confirms the diagnosis: at every anchor weight relevance is lower than with the plain share, and at $\beta = 0.1$ the policy loses 70% of NDCG@10 instead of 35%.

Table 7: Signals of the training reward model on the 100 best candidates of every KION test user (run E). Spread is a standard deviation. *Relevance AUC* is the probability that the candidate the user really consumed next has a higher value than the other candidates.

Signal	Spread across candidates	Spread across users	Relevance AUC
Consumed share $\hat{\rho}$	0.155	0.157	0.535
Session continuation $\hat{c}$	0.051	0.101	0.505
Return $\hat{u}$	0.031	0.126	0.546
Choice $\hat{q}$	0.141	0.116	0.560
$\hat{q} \cdot \hat{\rho}$	0.128	0.145	0.541
Log-probability of the recommender	0.700	0.154	0.701

5.6 Long-term signals: predictable, but barely controlled by the item

Predictability. Both retention signals are predicted clearly better than chance on KION. The AUC of session continuation is 0.66 on the validation and the test period; that of return is 0.70 on validation and 0.72 on test (Table 20 in the appendix). In the test period the outcome of return is known for only 26 thousand of 184 thousand events, because the one-week window is as long as the period, so the validation period carries the evidence.

Leverage. Predictability does not make a signal a useful reward. For the 100 best candidates of every test user, Table 7 shows how much each signal varies across the candidates of one user, which is all a policy can use, and across users. The consumed share varies as much across candidates as across users. Return varies four times less across candidates: 0.03 against 0.13. Whether a user returns depends mainly on the user and hardly on what is shown.

Rewarding retention. Table 8 and Figure 5 show fine-tuning on rewards that contain the retention signals.

A reward for return is a weak lever, as the spread predicted. It raises the predicted probability of return by 1.9 points (from 0.713 to 0.732) for 7% of NDCG@10 at $\beta = 0.3$, and by 4.7 points for 23% at $\beta = 0.1$. The ordinary share reward raises it by 1.5 points at $\beta = 0.3$ on its own: according to the reward model, those who finish come back more often. A dedicated reward adds less than half a point to that.

A reward for session continuation is harmful. At $\beta = 0.1$ the predicted continuation rises from 0.35 to 0.61 while the predicted share falls from 0.46 to 0.16 and the predicted watch time almost sixfold. The policy recommends titles that users abandon: after an abandoned title a person starts another one the same day, and the session "continues". On KION this signal rewards dissatisfaction.

Return in the reward makes the lists more diverse. The pair overlap falls from 0.21 to 0.15 at $\beta = 0.3$ and to 0.09 at 0.1, and coverage rises. Added to the share, return acts as a brake: at $\beta = 0.1$ the combined reward loses 19% of NDCG@10 where the plain share loses 35%, and the share rises by 22% instead of 41%.

Table 8: Retention rewards on KION (run E), 1,000 PPO iterations each. Share, continuation and return are the predictions of the held-out reward model for the generated lists.

Reward	NDCG@10, all	NDCG@10, liked	Share $\hat{\rho}$	Contin. $\hat{c}$	Return $\hat{u}$	Coverage	Pair overlap
Pretrained recommender	0.1173	0.1214	0.459	0.348	0.713	0.495	0.209
Anchor weight $\beta = 0.3$							
Share	0.1048	0.1239	0.562	0.343	0.728	0.434	0.260
Session	0.0999	0.1017	0.439	0.397	0.739	0.558	0.107
Return	0.1088	0.1130	0.462	0.368	0.732	0.517	0.147
Share + return	0.1093	0.1211	0.510	0.356	0.729	0.488	0.188
Share + session + return	0.1072	0.1160	0.494	0.367	0.733	0.519	0.156
Anchor weight $\beta = 0.1$							
Share	0.0767	0.0980	0.645	0.382	0.745	0.274	0.284
Session	0.0245	0.0287	0.161	0.609	0.769	0.385	0.334
Return	0.0903	0.0945	0.466	0.393	0.760	0.551	0.091
Share + return	0.0952	0.1139	0.559	0.365	0.749	0.467	0.186
Share + session + return	0.0874	0.1020	0.534	0.395	0.764	0.536	0.113

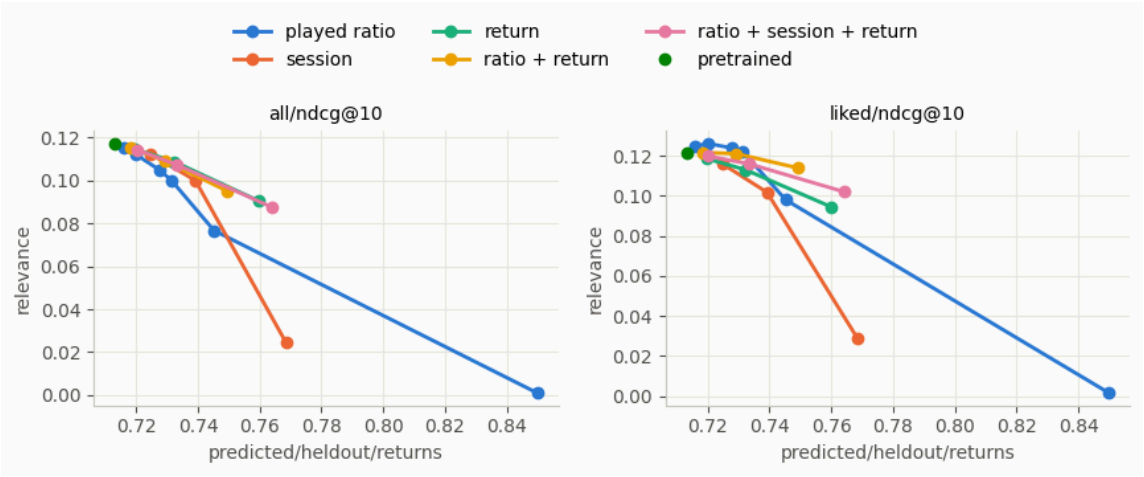


Figure 5: Relevance against the predicted probability of return on KION (run E), on all future items (left) and on liked items (right). Each line is one reward at three anchor weights. The rightmost point is the plain share with no anchor: the collapsed policy receives the highest predicted probability of return.

All return values here are predictions of a model trained on observations. It knows that people returned more often after such items; it does not know that the item was the cause. Offline one can test whether a signal is predictable, whether it has leverage and what optimising it does to the lists. Whether users actually return can only be shown online.

5.7 Generating several steps ahead

Lists drift away from the real continuation. The probability that a generated item is in the user's real continuation falls along the list: on Yambda from 0.165 at the first position to 0.06 at the tenth for the pretrained recommender, and on KION from 0.088 to 0.016 (after RL: from 0.081 to 0.018). On KION generation falls behind ranking as the list grows, and more so after RL: NDCG@10 of 0.1173 against 0.1220 before, and 0.1039 against 0.1103 after (Table 21 in the appendix). On Yambda generation stays ahead of ranking after RL as well (0.119 against 0.115).

The reward model expects less further down the list, unless the policy shapes its context. For the pretrained recommender the predicted share falls with the position on both datasets (on Yambda from 0.67 to 0.55; on KION from 0.53 to 0.38 in run C). On KION the fine-tuned policy keeps this profile at a higher level, from 0.62 to 0.43, and its advantage shrinks from ten points at the first position to five at the tenth. The two reward models disagree by 0.05–0.06 at every position for both policies (Figure 6): RL does not take the list to where they part. For the collapsed Yambda policy the picture is the opposite: the predicted share *rises* along the list, from 0.75 to more than 0.82, and the two reward models diverge more from the fourth position on.

Part of the profile is made by the simulation. A generated item has no observed feedback, but the reward model needs one to score the next item. Figure 7 scores the same lists of the fine-tuned policy under three assumptions: the model is fed its own expectation, a random outcome (finished with probability equal to the expectation, skipped otherwise), or "everything was finished". The first position depends only on the real history and is the same in all three. At the tenth position on KION the predicted share is 0.43, 0.53 or 0.68, a spread of 25 points; for the PPO policy on Yambda it is 0.56, 0.70 or 0.90, a spread of about 35 points; for the collapsed Yambda policy it is 0.82, 0.91 or 0.96. The expectation is the most pessimistic assumption in all cases, probably because of the input it creates: real consumed shares are close to 0 or to 1, and the reward model has hardly seen a history of items each consumed about halfway. Since RL was trained with this assumption, both the level of its multi-step reward and the decline along the list are partly its artefacts. The reward model has also learned that a user who has just finished several items will finish the next, so a policy that picks items with a high predicted share builds itself a favourable context. A multi-step predicted reward from this simulator cannot be reported without stating the assumption.

5.8 Does it take RL?

Table 9 and Figure 8 place the four alternatives of Section 3.6 next to RL on KION.

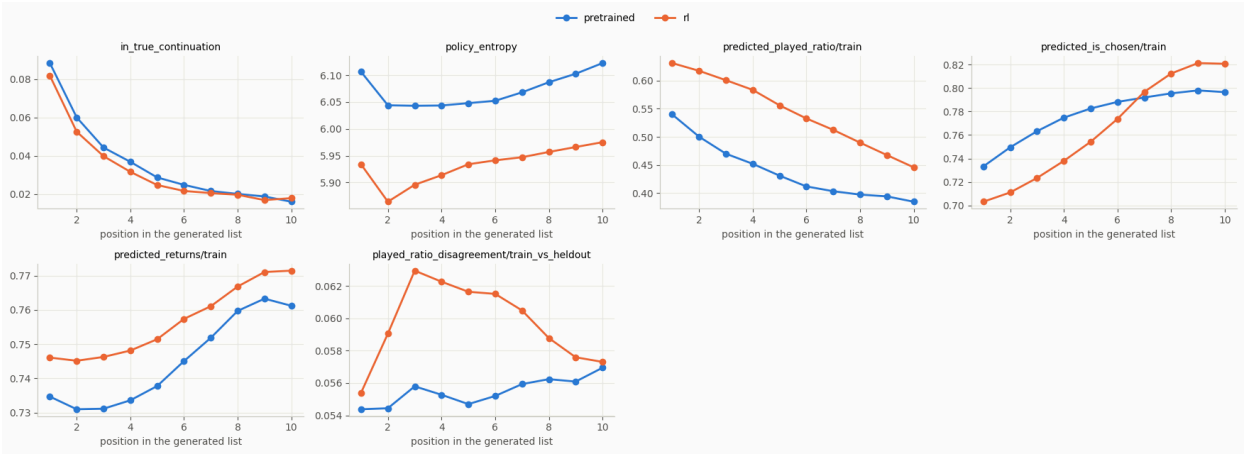
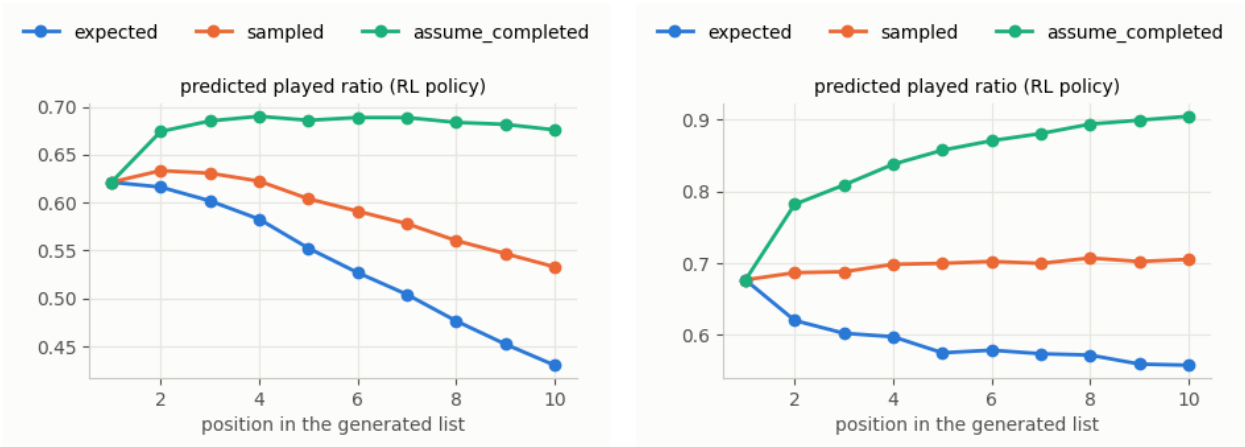


Figure 6: Generated lists by position on KION (run E): the pretrained recommender against the main RL run. Top row: probability that the item is in the user's real continuation, entropy of the policy, predicted share, and predicted choice probability. Bottom row: predicted probability of return, and the disagreement between the two reward models on the share. The choice and return signals rise along the list while real hits fall.



(a) KION, PPO (run C)

(b) Yambda, PPO (run D)

Figure 7: The share predicted by the training reward model for the lists of the fine-tuned policy under three assumptions about the feedback on the items generated earlier: the model's own expectation, a sampled outcome, and "everything was finished".

Re-ranking matches RL up to a reward gain of about one third. At $w = 0.3$ re-ranking reaches a predicted share of 0.554 with NDCG@10 of 0.112 overall and 0.134 on liked titles; the RL run closest in

Table 9: RL and its alternatives on KION test users (run E). κ is the cap on importance weights, w the weight of the recommender's log-probability in re-ranking.

Lists of ten	NDCG@10, all	NDCG@10, liked	Recall@10, skipped	Share (held-out)	Coverage	Pair overlap
Pretrained recommender, generated	0.1173	0.1214	0.1382	0.459	0.495	0.209
RL, $\beta = 0.3$ (main run)	0.1039	0.1230	0.1103	0.565	0.438	0.260
Liked-only fine-tuning	0.1036	0.1258	0.1017	0.552	0.473	0.242
Off-policy REINFORCE, $\kappa = 0.5$	0.1032	0.1237	0.1039	0.538	0.504	0.205
Off-policy REINFORCE, $\kappa = 2$	0.0901	0.1136	0.0854	0.564	0.424	0.197
Off-policy REINFORCE, $\kappa = 10$	0.0575	0.0737	0.0529	0.576	0.320	0.153
Re-ranking, $w = 1$	0.1199	0.1336	0.1375	0.503	0.564	0.147
Re-ranking, $w = 0.3$	0.1119	0.1343	0.1200	0.554	0.534	0.168
Re-ranking, $w = 0.1$	0.0944	0.1235	0.0883	0.610	0.449	0.197
Re-ranking, $w = 0$	0.0568	0.0782	0.0544	0.639	0.373	0.170
Best of 2 sampled lists	0.1079	0.1125	0.1357	0.479	0.506	0.195
Best of 8 sampled lists	0.1053	0.1133	0.1296	0.507	0.496	0.197
Best of 32 sampled lists	0.1021	0.1110	0.1259	0.526	0.498	0.199

reward (the main run, 0.565) has 0.104 and 0.123. At $w = 1$ re-ranking improves on the generated lists of the pretrained recommender on every axis at once. Up to a predicted share of about 0.62, re-ranking is at least as relevant as RL at equal reward. Part of the advantage is the starting point: re-ranking starts from the ranked list, which on KION is 0.005 NDCG@10 better than the generated one, and the ranked lists of the RL policy itself reach 0.110 overall (run C).

Liked-only fine-tuning lands on the main RL run. One epoch of supervised training on items watched at least half gives the same overall NDCG@10 (0.104), a slightly higher relevance on liked titles, a lower recall of abandoned ones and a predicted share of 0.552 against 0.565. It needs no reward model. In run C this stage together with all of re-ranking took 3.4 minutes, against 14.3 for the main RL run.

Off-policy REINFORCE at the cap of 0.5 lands there too, and keeps diversity. NDCG@10 is the same (0.103), NDCG@10 on liked titles is 1.9% above the pretrained recommender, and the reward gain is +17% against +23% for RL. Coverage and pair overlap stay where the pretrained recommender had them (0.504 and 0.205), while RL narrows the lists (0.438 and 0.260). Raising the cap is costly: at 10, 51% of NDCG@10 is lost for +26% of reward. The direction is that of [9], where raising the cap from 0.5 to 10 cost 9% of recall, but the effect is much stronger here. At no cap does the policy converge to a few items: at 10 the coverage is still 0.32.

The best of several sampled lists is the weakest method. Sampling already costs relevance (0.108 with two lists against 0.117 for the greedy list), and the reward grows slowly: +4% with two lists, +15% with thirty-two.

RL reaches further. Re-ranking 100 candidates cannot lift the predicted share above 0.64, and close to that limit its relevance drops sharply. RL at $\beta = 0.1$ reaches 0.645 with NDCG@10 of 0.077, above the 0.057 of re-ranking at its limit. This is the one region where RL is the better tool, and it is the region where relevance on liked titles is already a fifth lower.

The probable reason is the reward. The predicted share is a sum of terms for individual items, and on KION it is mostly a property of the title. Choosing items with a high predicted reward one at a time is then nearly optimal, which is what re-ranking does. RL should win when the reward depends on the list as a whole, and this reward depends on the list only through the simulated feedback on earlier items.

On Yambda every method in the safe region yields a reward gain of 1% to 2.5% (Table 22 in the appendix). Liked-only fine-tuning again lands on the main RL run (NDCG@10 0.118 against 0.119, predicted share 0.615 against 0.617). Re-ranking covers a region that RL did not reach with this grid, up to +6.5% of reward, at a cost in relevance. At an equal predicted share of 0.625 the generated lists of RL are more relevant than the re-ranked ones (0.117 against about 0.113), but the ranked lists of the same policy give 0.114: the difference comes from generation being better than ranking on Yambda, not from RL.

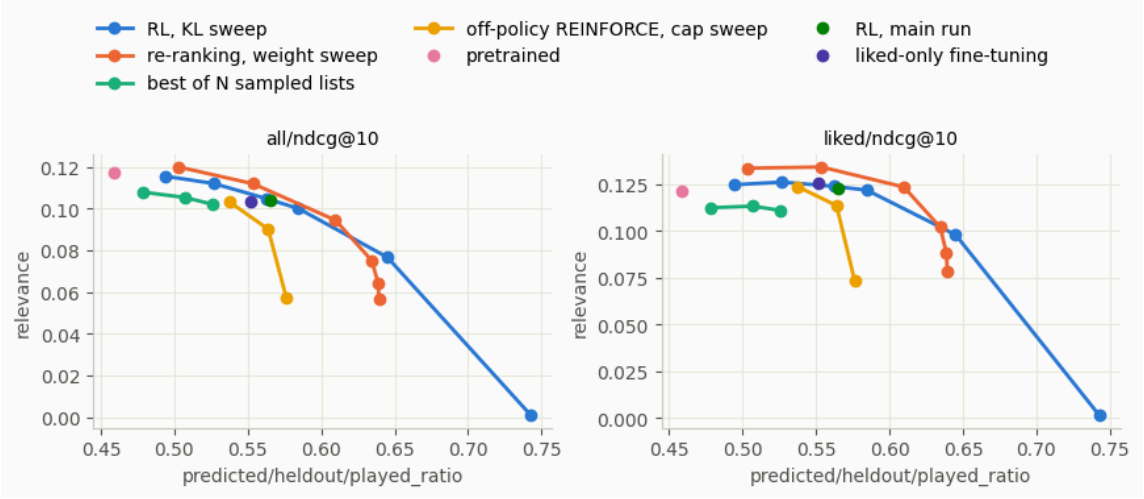


Figure 8: Relevance against the share predicted by the held-out reward model on KION (run E), on all future items (left) and on liked items (right): RL under the anchor sweep, re-ranking under the weight sweep, off-policy REINFORCE under the cap sweep, the best of n sampled lists, the main RL run, liked-only fine-tuning and the pretrained recommender.

Table 10: How the three settings differ. Figures for [8] are NDCG@10 on MovieLens-1M and Steam-2M across their settings; figures for [9] are from their Tables 1 and 3. Our figures are judged by the held-out reward model.

	Petrov and Macdonald [8]	**Matveev et al. [9]**	**This work**
Reward	NDCG against the real next item plus diversity or a popularity penalty	Logged like and played share of logged recommendations	Share predicted by a learned model
The policy is updated on	Its own generated lists	Logged sessions, importance weights capped at 0.5	Its own generated lists
What keeps relevance	The NDCG term inside the reward	The policy can only re-weight logged actions	The KL penalty towards the pretrained recommender
Role of a learned reward model	None	Evaluation only	Training signal
Relevance after RL	NDCG@10 from +3% to −38% depending on the weight of the second term	Recall@100 −2%; −6% on the actions with the lowest return	NDCG@10 at $\beta = 0.3$: −2% (Yambda, REINFORCE), +2% (Yambda, PPO), −11% (KION). At 0.1: −89%, +1%, −35%
Reward after RL	The target metric improves	Model-based return +2% to +4%	Predicted share at $\beta = 0.3$: +3% (Yambda, REINFORCE), +1% (Yambda, PPO), +23% (KION)

6 Discussion

6.1 Relation to the two studies this work builds on

Both studies report RL that keeps relevance. Here the first main run did not keep it, the KION run kept it at the price of 11% of overall NDCG@10, and the PPO run on Yambda changed almost nothing. The difference from those studies is not the optimiser but where the reward comes from and on which data the policy is updated (Table 10).

Three conclusions follow. First, in a regime like theirs, with a strong anchor, the REINFORCE result on Yambda agrees with Matveev et al. almost number for number: a loss of relevance of about 2%, a loss about three times larger on interactions users did not want, and a reward gain of a few per cent. KION shows the same shape at a larger scale: −11% overall, −20% on abandoned titles, no loss on finished ones, +23% of reward. Second, both studies also lose relevance when the constraint is relaxed. Petrov and Macdonald lose 15% of NDCG@10 on MovieLens-1M and 38% on Steam-2M at their strongest popularity penalty; Matveev et al. lose 9% of recall when the cap on importance weights is raised from 0.5 to 10. What cannot happen to them is a collapse to ten items: in one case relevance to real data is inside the reward, in the other the policy never leaves logged actions. The setting of this paper, in which a learned model is the only source of reward, removes both protections. Third, the optimiser of Petrov and Macdonald is not what protects them: PPO without an anchor collapsed like REINFORCE, and with an anchor its clipping never engaged.

The reward study tested both protections directly on KION. The reward of [8] does keep relevance without an anchor and, as its authors found, gives no notable gain in NDCG; added to the predicted share it keeps relevance better than the anchor. What their paper does not show and ours does is that without a constraint on diversity the policy contracts onto popular items. The method of [9] also behaves as described: at a cap of 0.5 the shift is mild and diversity is kept, relevance is lost as the cap grows, and the policy never converges to a few items. Both protections work, and each protects its own thing: the logged reward protects hits, learning on the log protects the distribution.

6.2 Three ways in which optimising the reward goes wrong

The failures observed here have three distinct mechanisms, and they call for different remedies.

Exploitation of model error. The policy finds items for which the reward model errs upwards. The signature is a gap between the two reward models: +40% against +16% on Yambda under a weak anchor. The mechanism is selection: the log holds only what users chose, so for an item this user would never choose the model falls back on what it knows about the item on average, and a niche item with a devoted audience looks good for everybody. A held-out copy detects it; accuracy on logged events does not prevent it.

A misspecified reward. The model is right and the reward asks for the wrong thing. On KION the share of a series is taken of the whole series, so a policy rewarded for the share moves from series to films, the held-out model confirms the gain, and the predicted watch time falls. The time variant errs in the opposite direction, and the session reward promotes abandoned titles. A second reward model cannot detect this; only inspecting what the policy moved to does.

A simulation artefact. The reward of late positions depends on the feedback assumed for the earlier ones, by 25 to 35 points here. Neither a second model nor relevance metrics expose it; scoring the same lists under several assumptions does.

6.3 Practical implications

Five checks follow for anyone fine-tuning a recommender against a learned reward. (i) Fit a second reward model on other users and report every gain from both; split relevance by the logged outcome as a model-free check. (ii) Do not reuse an anchor weight across estimators, rewards or datasets; report the KL divergence a run ended at. (iii) Before rewarding a signal, measure its spread across the candidates of one user against its spread across users. A signal that varies only across users cannot be moved by choosing items. (iv) State the feedback assumption with any multi-step predicted reward. (v) Compare with re-ranking by the same reward model before training anything: for an additive, item-determined reward it sets the frontier up to a substantial gain.

7 Limitations

- **One seed per configuration.** Run E repeated the main KION result with different reward models but the same recommender. Intervals cover the sample of users, not the variation between training runs, and the baselines have no intervals.
- **The estimator explanation is a hypothesis.** The learning rate and the number of lists per step changed together with the advantage estimator.
- **The reward study covers KION only.** The Yambda PPO run used an earlier version of the pipeline, without retention signals, logged rewards and off-policy REINFORCE.
- **The anchor grids missed both transitions on Yambda:** between 0.1 and 0.3 for REINFORCE, between 0 and 0.1 for PPO.
- **Baselines are untuned,** and GRU4Rec and BERT4Rec on Yambda were stopped by the epoch limit and not by their validation loss.
- **The choice head is one construction.** It was trained on popularity-distributed replacements and stopped before convergence; its failure speaks about this construction, not about every way of teaching a reward model relevance.
- **Return is evaluated mostly on the validation period,** which also selected the stopping epoch; in the test period the outcome is known for 14% of events.
- **PPO's clipping almost never engaged,** so no run tests the part of PPO that distinguishes it.
- **Re-ranking and RL start from different lists:** re-ranking from ranked lists, RL from generated ones. On KION this favours re-ranking, on Yambda RL.
- **On KION the watched share is tied to the length of the title,** and with it the reward and the split into liked and skipped items.
- **Advantages are normalised,** which neither of the two studies we build on does; anchor weights are comparable neither with their scales nor across our settings.
- **Everything is offline.** No number here says how real users would respond to a fine-tuned policy.

8 Conclusion

We fine-tuned a generative recommender against a learned reward model in the setting with the fewest protections: the model is the only source of reward, and a KL anchor to the pretrained recommender is the only constraint. Unconstrained, the policy always collapses, and a reward model that is accurate and calibrated on logged events does not prevent it. With an anchor that holds, the gain depends on how much of the reward is a property of the item and not of the user: large and confirmed on KION, negligible on Yambda. The anchor weight means different things under different advantage estimators. Relevance can be required in three places, and each preserves something different: the anchor preserves the distribution

of the pretrained recommender, a logged-relevance reward preserves hits but not diversity, and learning from logged actions preserves both for a smaller gain. Retention signals can be predictable without being controllable by the choice of item, and a multi-step predicted reward is partly a property of the simulation. For an additive, item-determined reward, re-ranking with the same reward model reaches the same frontier as RL up to a gain of about one third.

Several steps follow directly. The logged-relevance reward and a weak anchor fail in complementary ways, so their combination is the main candidate for a method that beats both the anchor and re-ranking. The normalisation hypothesis has a direct test: PPO with a leave-one-out baseline at the same anchor weights, with a finer grid below 0.1. The reward study should be repeated on Yambda, where the reward is determined more by the user than by the item and the conclusion about weak leverage may be starker. A choice head with negatives drawn from the recommender's own candidates, stopped on its own quality, would test whether the reward model can be taught relevance at all. A consumed share measured relative to what is usual for the item or its type would remove the dependence on length. Finally, every main result needs several seeds.

Code availability

The code that produced every table and figure, with the per-run reports, is in the repository `github.com/daniilantonovv/RecSysMaxxing`.

References

[1] W.-C. Kang and J. McAuley, "Self-Attentive Sequential Recommendation," *arXiv preprint arXiv:1808.09781*, 2018.

[2] A. V. Petrov and C. Macdonald, "Generative Sequential Recommendation with GPTRec," *arXiv preprint arXiv:2306.11114*, 2023.

[3] S. Rajput *et al.*, "Recommender Systems with Generative Retrieval," *arXiv preprint arXiv:2305.05065*, 2023.

[4] J. Zhai *et al.*, "Actions Speak Louder than Words: Trillion-Parameter Sequential Transducers for Generative Recommendations," *arXiv preprint arXiv:2402.17152*, 2024.

[5] J. Deng *et al.*, "OneRec: Unifying Retrieve and Rank with Generative Recommender and Iterative Preference Alignment," *arXiv preprint arXiv:2502.18965*, 2025.

[6] M. Chen, A. Beutel, P. Covington, S. Jain, F. Belletti, and E. Chi, "Top-K Off-Policy Correction for a REINFORCE Recommender System," *arXiv preprint arXiv:1812.02353*, 2018.

[7] E. Ie *et al.*, "Reinforcement Learning for Slate-based Recommender Systems: A Tractable Decomposition and Practical Methodology," *arXiv preprint arXiv:1905.12767*, 2019.

[8] A. Petrov and C. Macdonald, "Aligning GPTRec with Beyond-Accuracy Goals with Reinforcement Learning," *arXiv preprint arXiv:2403.04875*, 2024.

[9] A. Matveev, S. Makeev, A. Krasilnikov, V. Baikalov, S. Liamaev, and K. Khrylchenko, "Session-Level Optimization for Large-Scale Retrieval using REINFORCE with Multi-Step Off-Policy Correction," *arXiv preprint arXiv:2607.02818*, 2026.

[10] T. Yu *et al.*, "MOPO: Model-based Offline Policy Optimization," *arXiv preprint arXiv:2005.13239*, 2020.

[11] R. Kidambi, A. Rajeswaran, P. Netrapalli, and T. Joachims, "MOReL: Model-Based Offline Reinforcement Learning," *arXiv preprint arXiv:2005.05951*, 2020.

[12] M. Janner, J. Fu, M. Zhang, and S. Levine, "When to Trust Your Model: Model-Based Policy Optimization," *arXiv preprint arXiv:1906.08253*, 2019.

[13] L. Gao, J. Schulman, and J. Hilton, "Scaling Laws for Reward Model Overoptimization," *arXiv preprint arXiv:2210.10760*, 2022.

[14] T. Coste, U. Anwar, R. Kirk, and D. Krueger, "Reward Model Ensembles Help Mitigate Overoptimization," *arXiv preprint arXiv:2310.02743*, 2023.

[15] J. Skalse, N. H. R. Howe, D. Krasheninnikov, and D. Krueger, "Defining and Characterizing Reward Hacking," *arXiv preprint arXiv:2209.13085*, 2022.

[16] C. Gao *et al.*, "Alleviating Matthew Effect of Offline Reinforcement Learning in Interactive Recommendation," *arXiv preprint arXiv:2307.04571*, 2023.

[17] C. Gao *et al.*, "CIRS: Bursting Filter Bubbles by Counterfactual Interactive Recommender System," *arXiv preprint arXiv:2204.01266*, 2022.

[18] R. Jiang, S. Chiappa, T. Lattimore, A. György, and P. Kohli, "Degenerate Feedback Loops in Recommender Systems," *arXiv preprint arXiv:1902.10730*, 2019.

[19] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, and I. Sutskever, "Language Models are Unsupervised Multitask Learners," technical report, 2019.

[20] B. Hidasi, A. Karatzoglou, L. Baltrunas, and D. Tikk, "Session-based Recommendations with Recurrent Neural Networks," *arXiv preprint arXiv:1511.06939*, 2015.

[21] F. Sun *et al.*, "BERT4Rec: Sequential Recommendation with Bidirectional Encoder Representations from Transformer," *arXiv preprint arXiv:1904.06690*, 2019.

[22] E. Botta *et al.*, "PinRec: Unified Generative Retrieval for Pinterest Recommender Systems," *arXiv preprint arXiv:2504.10507*, 2025.

[23] R. Deffayet, T. Thonet, J.-M. Renders, and M. de Rijke, "Generative Slate Recommendation with Reinforcement Learning," *arXiv preprint arXiv:2301.08632*, 2023.

[24] X. Xin, A. Karatzoglou, I. Arapakis, and J. M. Jose, "Self-Supervised Reinforcement Learning for Recommender Systems," *arXiv preprint arXiv:2006.05779*, 2020.

[25] X. Xin, A. Karatzoglou, I. Arapakis, and J. M. Jose, "Supervised Advantage Actor-Critic for Recommender Systems," *arXiv preprint arXiv:2111.03474*, 2021.

[26] K. Christakopoulou *et al.*, "Reward Shaping for User Satisfaction in a REINFORCE Recommender," *arXiv preprint arXiv:2209.15166*, 2022.

[27] E. Ie *et al.*, "RecSim: A Configurable Simulation Platform for Recommender Systems," *arXiv preprint arXiv:1909.04847*, 2019.

[28] J.-C. Shi, Y. Yu, Q. Da, S.-Y. Chen, and A.-X. Zeng, "Virtual-Taobao: Virtualizing Real-world Online Retail Environment for Reinforcement Learning," *arXiv preprint arXiv:1805.10000*, 2018.

[29] K. Zhao *et al.*, "KuaiSim: A Comprehensive Simulator for Recommender Systems," *arXiv preprint arXiv:2309.12645*, 2023.

[30] J. Schulman, S. Levine, P. Moritz, M. I. Jordan, and P. Abbeel, "Trust Region Policy Optimization," *arXiv preprint arXiv:1502.05477*, 2015.

[31] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal Policy Optimization Algorithms," *arXiv preprint arXiv:1707.06347*, 2017.

[32] N. Jaques, S. Gu, D. Bahdanau, J. M. Hernández-Lobato, R. E. Turner, and D. Eck, "Sequence Tutor: Conservative Fine-Tuning of Sequence Generation Models with KL-control," *arXiv preprint arXiv:1611.02796*, 2016.

[33] N. Jaques *et al.*, "Way Off-Policy Batch Deep Reinforcement Learning of Implicit Human Preferences in Dialog," *arXiv preprint arXiv:1907.00456*, 2019.

[34] D. M. Ziegler *et al.*, "Fine-Tuning Language Models from Human Preferences," *arXiv preprint arXiv:1909.08593*, 2019.

[35] L. Ouyang *et al.*, "Training language models to follow instructions with human feedback," *arXiv preprint arXiv:2203.02155*, 2022.

[36] Y. Wu, G. Tucker, and O. Nachum, "Behavior Regularized Offline Reinforcement Learning," *arXiv preprint arXiv:1911.11361*, 2019.

[37] L. Zou, L. Xia, Z. Ding, J. Song, W. Liu, and D. Yin, "Reinforcement Learning to Optimize Long-term User Engagement in Recommender Systems," *arXiv preprint arXiv:1902.05570*, 2019.

[38] Q. Cai *et al.*, "Reinforcing User Retention in a Billion Scale Short Video Recommender System," *arXiv preprint arXiv:2302.01724*, 2023.

[39] T. A. Mann *et al.*, "Learning from Delayed Outcomes via Proxies with Applications to Recommender Systems," *arXiv preprint arXiv:1807.09387*, 2018.

[40] Y. Su *et al.*, "Long-Term Value of Exploration: Measurements, Findings and Algorithms," *arXiv preprint arXiv:2305.07764*, 2023.

[41] R. Zhan *et al.*, "Deconfounding Duration Bias in Watch-time Prediction for Video Recommendation," *arXiv preprint arXiv:2206.06003*, 2022.

[42] H. Zhao, L. Zhang, J. Xu, G. Cai, Z. Dong, and J.-R. Wen, "Uncovering User Interest from Biased and Noised Watch Time in Video Recommendation," *arXiv preprint arXiv:2308.08120*, 2023.

[43] S. Bengio, O. Vinyals, N. Jaitly, and N. Shazeer, "Scheduled Sampling for Sequence Prediction with Recurrent Neural Networks," *arXiv preprint arXiv:1506.03099*, 2015.

[44] M. Ranzato, S. Chopra, M. Auli, and W. Zaremba, "Sequence Level Training with Recurrent Neural Networks," *arXiv preprint arXiv:1511.06732*, 2015.

[45] C. Guo *et al.*, "PROMISE: Process Reward Models Unlock Test-Time Scaling Laws in Generative Recommendations," *arXiv preprint arXiv:2601.04674*, 2026.

[46] R. Deffayet, T. Thonet, J.-M. Renders, and M. de Rijke, "Offline Evaluation for Reinforcement Learning-based Recommendation: A Critical Issue and Some Alternatives," *arXiv preprint arXiv:2301.00993*, 2023.

[47] M. Dudik, J. Langford, and L. Li, "Doubly Robust Policy Evaluation and Learning," *arXiv preprint arXiv:1103.4601*, 2011.

[48] M. Ferrari Dacrema, P. Cremonesi, and D. Jannach, "Are We Really Making Much Progress? A Worrying Analysis of Recent Neural Recommendation Approaches," *arXiv preprint arXiv:1907.06902*, 2019.

[49] M. Gutmann and A. Hyvärinen, "Noise-Contrastive Estimation: A New Estimation Principle for Unnormalized Statistical Models," in *Proceedings of the Thirteenth International Conference on Artificial Intelligence and Statistics*, 2010, pp. 297–304.

[50] R. J. Williams, "Simple Statistical Gradient-Following Algorithms for Connectionist Reinforcement Learning," *Machine Learning*, vol. 8, no. 3–4, pp. 229–256, 1992.

[51] W. Kool, H. van Hoof, and M. Welling, "Buy 4 REINFORCE Samples, Get a Baseline for Free!," in *ICLR 2019 Workshop on Deep Reinforcement Learning Meets Structured Prediction*, 2019.

[52] J. Schulman, P. Moritz, S. Levine, M. Jordan, and P. Abbeel, "High-Dimensional Continuous Control Using Generalized Advantage Estimation," *arXiv preprint arXiv:1506.02438*, 2015.

[53] A. Ploshkin *et al.*, "Yambda-5B — A Large-Scale Multi-modal Dataset for Ranking And Retrieval," *arXiv preprint arXiv:2505.22238*, 2025.

[54] A. Petrov, I. Safilo, D. Tikhonovich, and D. Ignatov, "MTS Kion Implicit Contextualised Sequential Dataset for Movie Recommendation," *arXiv preprint arXiv:2209.00325*, 2022.

[55] I. Loshchilov and F. Hutter, "Decoupled Weight Decay Regularization," *arXiv preprint arXiv:1711.05101*, 2017.

A Run configurations

Table 11: The five completed runs. The horizon is 10 items everywhere; windows are 128 events on Yambda and 48 on KION.

	A	**B**	**C**	**D**	**E**
Dataset	Yambda-50M	Yambda-50M	KION	Yambda-50M	KION
GPU	RTX PRO 6000	Tesla T4	RTX PRO 6000	RTX PRO 6000	RTX PRO 6000
Catalogue	50,000	30,000	7,992	50,000	7,992
Users	8,711	8,669	256,407	8,711	256,407
Recommender epochs	15	6	11 (early stop)	10	11 (early stop)
RL algorithm	REINFORCE	REINFORCE	PPO	PPO	PPO
Main RL run	4,000 × 1,024 lists	1,500 × 256	2,000 × 512	2,000 × 512	2,000 × 512
Lists per history	4	4	1	1	1
Sweep runs	2,000 steps	500 steps	1,000 iterations	1,000 iterations	1,000 iterations
Anchor weight, main run	0.1	0.1	0.3	0.3	0.3
Anchor weights in the sweep	0, 0.01, 0.03, 0.1, 0.3, 1, 3	0, 0.03, 0.3, 3	0, 0.1, 0.2, 0.3, 0.5, 1	0, 0.1, 0.2, 0.3, 0.5, 1	0, 0.1, 0.2, 0.3, 0.5, 1
RL learning rate	$3 \cdot 10^{-5}$	$3 \cdot 10^{-5}$	10^{-5}	10^{-5}	10^{-5}
Baselines, alternatives, intervals	no	no	yes	yes	yes
Reward study	no	no	no	no	yes
Wall-clock time	9.5 h	5.0 h	1.3 h	6.0 h	3.1 h

Run B. The reduced-budget run looked better at first sight: its main RL run lost 68% of NDCG@10 where run A lost 89%. It did far less work (6 epochs on half as many training windows with a smaller softmax, and about eleven times fewer generated lists in the main run), and its policy was stopped while still degrading: relevance was falling at the last measurement, and the KL divergence to the pretrained recommender was 1.7 nats and rising, where run A levelled off at 2.8. Its sweep agrees with run A where they overlap: after 500 steps the runs with weights 0 and 0.03 had collapsed (NDCG@10 0.0017 and 0.0043, pair overlap 1.00 and 0.98) and the run with 0.3 had held (NDCG@10 0.1237 against 0.1226 for its pretrained recommender, recall of skipped tracks −2.4%, predicted share +2.7% and +2.4% by the two reward models).

Table 12: Wall-clock time by stage. One PPO iteration of 512 lists takes 1.1 seconds on Yambda (catalogue 50,000, windows of 128) and 0.43 seconds on KION (catalogue 7,992, windows of 48).

Stage, minutes	**A**	**B**	**C**	**D**	**E**
Pretrained recommender	99	109	4	66	4
Reward models	6	22	1	6	4
Baselines			5	81	6
Main RL run	74	56	14	38	14
Anchor sweep	249	61	33	109	33
Alternatives to RL			3	6	16
Reward sweep	142	46	16	54	65
Relevance inside the reward					43
Everything else	1	6	3	1	3
Total	570	300	79	361	188

B Anchor sweeps

Table 13: Anchor sweep on Yambda with REINFORCE (run A): generated lists for test users after 2,000 RL steps.

Anchor weight β	**Base**	**0**	**0.01**	**0.03**	**0.1**	**0.3**	**1**	**3**
NDCG@10, all	0.1175	0.0007	0.0007	0.0006	0.0118	0.1155	0.1183	0.1176
NDCG@10, liked	0.1010	0.0007	0.0007	0.0005	0.0114	0.0998	0.1021	0.1014
Recall@10, skipped	0.0737	0.0002	0.0002	0.0001	0.0059	0.0691	0.0734	0.0734
Predicted share, training RM	0.580	0.830	0.830	0.836	0.806	0.611	0.591	0.585
Predicted share, held-out RM	0.613	0.728	0.728	0.701	0.712	0.633	0.619	0.615
Catalogue coverage	0.523	0.0002	0.0002	0.0004	0.046	0.472	0.514	0.520
Pair overlap	0.001	1.000	1.000	0.895	0.596	0.005	0.002	0.002
Mean duration (s)	203	119	119	123	125	189	199	202
Share from the history	0.505	0.005	0.005	0.003	0.034	0.479	0.505	0.507

Table 14: Anchor sweep on Yambda with PPO (run D): generated lists for test users after 1,000 iterations.

Anchor weight β	**Base**	**0**	**0.1**	**0.2**	**0.3**	**0.5**	**1**
NDCG@10, all	0.1165	0.0011	0.1173	0.1177	0.1175	0.1179	0.1172
NDCG@10, liked	0.1012	0.0012	0.1020	0.1022	0.1015	0.1020	0.1015
Recall@10, skipped	0.0714	0.0002	0.0702	0.0727	0.0725	0.0725	0.0729
Predicted share, training RM	0.581	0.826	0.598	0.591	0.589	0.588	0.585
Predicted share, held-out RM	0.610	0.720	0.625	0.618	0.617	0.614	0.612
Catalogue coverage	0.519	0.0004	0.474	0.498	0.502	0.507	0.512
Pair overlap	0.0014	1.000	0.0054	0.0026	0.0023	0.0019	0.0017
Mean duration (s)	203	122	192	197	198	199	201
Share from the history	0.520	0.005	0.497	0.517	0.516	0.519	0.520

Table 15: Anchor sweep on KION with PPO (run C): generated lists for test users after 1,000 iterations.

Anchor weight β	**Base**	**0**	**0.1**	**0.2**	**0.3**	**0.5**	**1**
NDCG@10, all	0.1173	0.0007	0.0736	0.0987	0.1042	0.1114	0.1153
NDCG@10, liked	0.1214	0.0012	0.0944	0.1206	0.1240	0.1261	0.1250
Recall@10, skipped	0.1382	0.0004	0.0654	0.1006	0.1099	0.1280	0.1350
Predicted share, training RM	0.433	0.871	0.653	0.559	0.534	0.499	0.467
Predicted share, held-out RM	0.444	0.748	0.652	0.570	0.544	0.509	0.477
Predicted time, held-out RM (s)	6,361	4,511	4,101	5,675	5,775	6,006	6,098
Catalogue coverage	0.495	0.006	0.248	0.386	0.423	0.451	0.474
Pair overlap	0.209	0.991	0.377	0.317	0.280	0.250	0.226
Mean duration (hours)	5.7	1.7	1.9	3.0	3.3	4.1	4.8

Table 16: Replication on KION: the same recommender, reward models trained anew.

	Run C	**Run E**
Main RL run, $\beta = 0.3$		
NDCG@10, all	0.1039 (−11.4%)	0.1039 (−11.4%)
NDCG@10, liked	0.1228 (+1.2%)	0.1230 (+1.3%)
Recall@10, skipped	0.1112 (−19.6%)	0.1103 (−20.2%)
Predicted share, training RM	+23.9%	+24.0%
Predicted share, held-out RM	+22.8%	+23.1%
Catalogue coverage	0.434	0.438
Pair overlap	0.275	0.260
Mean duration of titles (hours)	3.2	3.3
Sweep runs		
$\beta = 0.1$: NDCG@10, all	0.0736	0.0767
$\beta = 0.1$: predicted share, held-out RM	+46.9%	+40.6%
No anchor: distinct titles in all lists	46	81

C Reward variants

In the tables of this section *time* multiplies the predicted share by the relative duration of the item, and *session* is the predicted probability that the session continues: another listen within 30 minutes on Yambda,

another title within 7 days on KION (run C). The Yambda REINFORCE variants ran at an anchor weight that the sweep later placed on the collapsing side, so Table 17 shows in which direction each reward is gamed and is not a fair comparison of rewards.

Table 17: Reward variants on Yambda with REINFORCE (run A), 2,000 steps each at $\beta = 0.1$.

Reward	Base	Share	Time	Share + session	Session only
NDCG@10, all	0.1175	0.0118	0.0003	0.0105	0.0056
Predicted share (held-out)	0.613	0.712	0.545	0.710	0.634
Predicted seconds (held-out)	123	89	801	89	110
Predicted continuation (held-out)	0.937	0.941	0.869	0.941	0.941
Catalogue coverage	0.523	0.046	0.0004	0.040	0.025
Pair overlap	0.001	0.596	0.960	0.555	0.111
Mean duration (s)	203	125	1,513	125	180

Table 18: Reward variants on Yambda with PPO (run D), 1,000 iterations each at $\beta = 0.3$.

Reward	Base	Time	Share + session	Session only
NDCG@10, all	0.1165	0.0910	0.1173	0.1164
NDCG@10, liked	0.1012	0.0783	0.1015	0.1006
Predicted share (held-out)	0.610	0.553	0.614	0.610
Predicted seconds (held-out)	122	364	121	122
Predicted continuation (held-out)	0.9371	0.9023	0.9372	0.9372
Catalogue coverage	0.519	0.339	0.509	0.513
Pair overlap	0.0014	0.134	0.0020	0.0016
Mean duration (s)	203	710	200	203

Table 19: Reward variants on KION with PPO (run C), 1,000 iterations each at $\beta = 0.3$. None of them collapsed.

Reward	Base	Share	Time	Share + session	Session only
NDCG@10, all	0.1173	0.1042	0.0885	0.1098	0.1093
NDCG@10, liked	0.1214	0.1240	0.0901	0.1217	0.1131
Recall@10, skipped	0.1382	0.1099	0.1042	0.1281	0.1324
Predicted share (held-out)	0.444	0.544	0.307	0.487	0.440
Predicted hours (held-out)	1.77	1.60	10.51	1.62	1.61
Predicted continuation (held-out)	0.804	0.805	0.763	0.812	0.821
Catalogue coverage	0.495	0.423	0.274	0.487	0.505
Pair overlap	0.209	0.280	0.304	0.189	0.144
Mean duration (hours)	5.7	3.3	63.6	4.3	5.1

On KION, adding the session signal to the share halves the shift: the share rises by 10% instead of 23%, relevance on liked titles is unchanged and overall NDCG@10 falls by 6%. The session-only reward raises the predicted probability of another viewing within a week from 0.804 to 0.821 and leaves the share where it was, at a cost of 7% of NDCG@10; it is the one variant that makes the lists less alike (pair overlap from 0.21 to 0.14).

D Further tables and figures

Table 20: Prediction of the retention signals by the training reward model on KION (run E). The held-out model differs by at most 0.004.

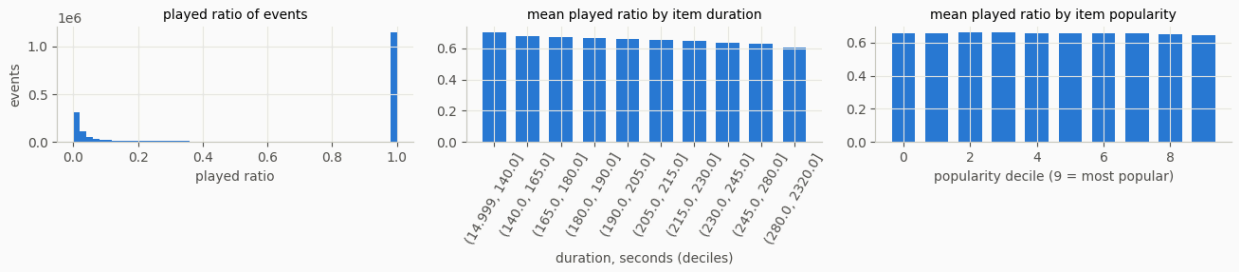
Signal	Period	AUC	Base rate	Events with a known outcome
Session continuation	validation	0.660	0.427	187,398
Session continuation	test	0.661	0.421	184,209
Return	validation	0.696	0.720	187,398
Return	test	0.719	0.702	26,004

Table 21: Ranking against item-by-item generation on KION test users (run C). The first item is the same by construction.

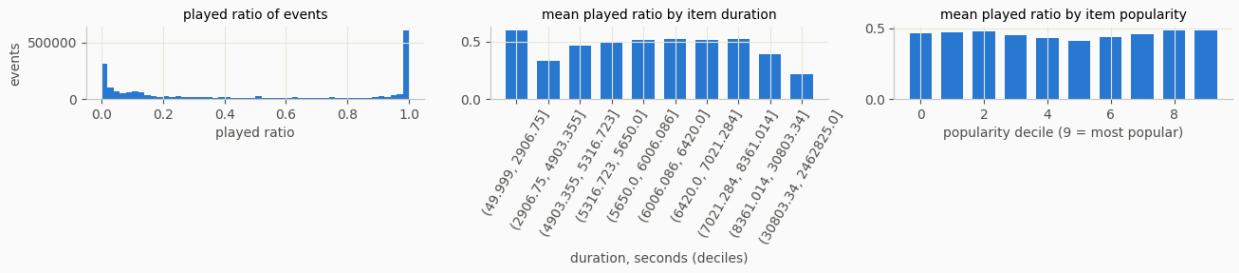
NDCG at	Base, ranked	Base, generated	RL, ranked	RL, generated
1	0.0885	0.0885	0.0812	0.0812
5	0.1055	0.1044	0.0943	0.0911
10	0.1220	0.1173	0.1103	0.1039

Table 22: RL against liked-only fine-tuning and re-ranking of 100 candidates on Yambda test users (run D).

Lists of ten	NDCG@10, all	NDCG@10, liked	Recall@10, skipped	Share (held-out)	Coverage	From history
Pretrained recommender, generated	0.1165	0.1012	0.0714	0.610	0.519	0.520
Pretrained recommender, ranked	0.1136	0.0989	0.0732	0.615	0.464	0.423
RL, $\beta = 1$	0.1172	0.1015	0.0729	0.612	0.512	0.520
RL, $\beta = 0.5$	0.1179	0.1020	0.0725	0.614	0.507	0.519
RL, $\beta = 0.3$ (main run)	0.1190	0.1032	0.0745	0.617	0.503	0.523
RL, $\beta = 0.2$	0.1177	0.1022	0.0727	0.618	0.498	0.517
RL, $\beta = 0.1$	0.1173	0.1020	0.0702	0.625	0.474	0.497
Liked-only fine-tuning	0.1183	0.1029	0.0720	0.615	0.512	0.522
Re-ranking, $w = 1$	0.1141	0.0993	0.0735	0.617	0.461	0.424
Re-ranking, $w = 0.3$	0.1142	0.0998	0.0733	0.621	0.456	0.423
Re-ranking, $w = 0.1$	0.1114	0.0975	0.0713	0.630	0.441	0.408
Re-ranking, $w = 0.03$	0.0941	0.0833	0.0585	0.642	0.400	0.350
Re-ranking, $w = 0$	0.0449	0.0384	0.0281	0.650	0.330	0.226



(a) Yambda-50M, catalogue of 50,000 tracks



(b) KION

Figure 9: The reward signal in the two processed logs. Left: distribution of the consumed share. Middle and right: mean consumed share by decile of item duration and of item popularity.

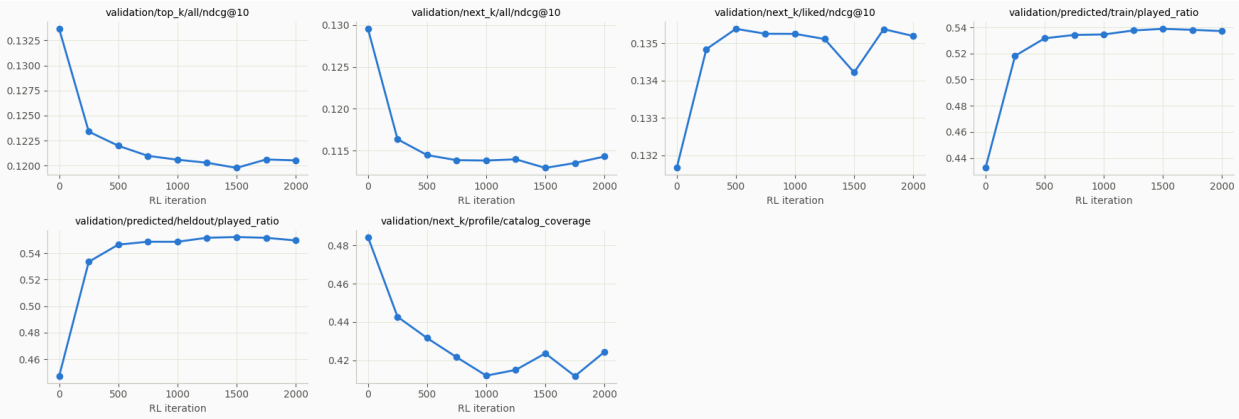


Figure 10: Greedy lists for validation users during the main RL run on KION (PPO, $\beta = 0.3$, run C). Over the first 500 iterations relevance on all titles falls and relevance on liked titles rises; the reward by both models flattens at the same time.